

به نام خدا

گزارش تمرین دوم درس سیستم های نهفته و بی درنگ



اعضای گروه:

سید محمد جزایری - 810101399

امیرارسلان شهبازی - 810101451

سید محمدحسین مظهری - 810101520

سید نوید هاشمی - 810101549

فهرست مطالب

2.....	فهرست مطالب
3.....	مقدمه
3.....	نحوه شکست کارهای پروژه بین اعضای گروه
5.....	بخش سمت اندروید
5.....	بررسی اجمالی
5.....	بخش اول: کدنویسی
6.....	فایل MainActivity.java
24.....	فایل NetworkClient.java
28.....	بخش دوم: رابط کاربری
29.....	صفحه‌ی اصلی برنامه یا mainScreen
31.....	صفحه‌ی کالیبراسیون یا calibrationScreen
33.....	بخش سمت رایانه شخصی
35.....	ابزار Profile and Trace
35.....	بخش سوالات
53.....	جمع‌بندی و نتیجه‌گیری

مقدمه

در این پروژه، یک سیستم Air Mouse طراحی و پیاده‌سازی شده است که هدف آن تبدیل گوشی هوشمند به یک کنترل‌کننده حرکتی برای ماوس رایانه است. در این سیستم، داده‌های خام سنسورهای گوشی شامل ژيروسکوپ، شتاب‌سنج و مغناطیس‌سنج دریافت شده و پس از کالیبراسیون و پردازش، به فرمان‌هایی مانند حرکت مکان‌نما، کلیک و اسکرول تبدیل می‌شوند. سپس این فرمان‌ها از طریق ارتباط UDP به رایانه ارسال شده و در سمت سیستم مقصد به رویدادهای واقعی ماوس تبدیل می‌گردند.

گزارش حاضر بخش‌های مختلف پروژه را بررسی می‌کند؛ از جمله پیاده‌سازی سمت اندروید در فایل‌های `MainActivity.java` و `NetworkClient.java`، طراحی رابط کاربری برنامه، اسکریپت سمت رایانه شخصی برای دریافت داده‌ها و اجرای فرمان‌های ماوس، و در نهایت تحلیل عملکرد برنامه با استفاده از ابزارهای `profiling` و `tracing`. هدف گزارش این است که هم ساختار پیاده‌سازی پروژه توضیح داده شود و هم تصمیم‌های فنی استفاده‌شده برای کاهش تأخیر، افزایش پایداری و بهبود تجربه کاربر بررسی شوند.

نحوه شکست کارهای پروژه بین اعضای گروه

شرح وظایف و مشارکت‌های ۴ عضو گروه به تفکیک به شرح زیر است:

سید محمد جزایری - 810101399

نقش اصلی: تحلیل‌گر سیستم (Profiling & Tracing) و مستندسازی عملکرد

- **راه‌اندازی ابزار Perfetto:** پیکربندی و راه‌اندازی کامل ابزار Profiling شرکت گوگل جهت ارزیابی دقیق عملکرد برنامه در سطح سیستم‌عامل.
- **تولید Trace و مدیریت رویدادها:** آماده‌سازی فایل `config.pbtx`، جاگذاری متدهای `Trace.beginSection` در بلوک‌های حیاتی کد اندروید و ثبت Trace‌های سیستم.
- **تحلیل و مستندسازی:** استخراج و تحلیل گراف‌های تولید شده (زمان‌بندی Thread‌ها، بررسی Context Switch‌ها)، استدلال علمی روی نتایج، پاسخ به ۱۱ سوال تئوری فاز پایانی، و نگارش کامل گزارش مربوط به بخش Profiling و Trace.

امیرارسلان شهبازی - 810101451

نقش اصلی: توسعه‌دهنده هسته اندروید، سنسورها و مستندسازی فرانت‌اند

- **پیاده‌سازی پایه اپلیکیشن:** راه‌اندازی معماری برنامه اندروید، طراحی رابط کاربری (UI) مطابق با نیازها و متصل کردن المان‌های گرافیکی به کدهای منطقی.
- **مدیریت سنسورها:** پیاده‌سازی اولیه `SensorManager`، ثبت شنونده‌ها برای سنسورهای شتاب‌سنج،ژیروسکوپ و مغناطیس‌سنج و استخراج داده‌های خام آن‌ها.
- **چرخه تست و مستندسازی:** گرفتن خروجی‌های APK برای تست‌های مکرر، اعمال تغییرات مستقل روی منطق جاوا برای بهبود حرکت، و نگارش گزارش نهایی مربوط به بخش توسعه و منطق اندروید (به صورت مشترک با محمدحسین).

سید محمدحسین مظهري - 810101520

نقش اصلی: توسعه‌دهنده شبکه اندروید، یکپارچه‌سازی سنسورها و مستندسازی فرانت‌اند

- **توسعه لایه شبکه:** مشارکت در توسعه کلاس `NetworkClient.java`، ایجاد ساختار ارسال داده‌ها روی بستر UDP و مدیریت Thread‌های مرتبط با شبکه.
- **یکپارچه‌سازی ارتباطات:** همکاری در اتصال خروجی سنسورها به ماژول ارسال شبکه و مدیریت تأییدیه‌ها (ACK) برای اطمینان از ارسال موفق رویدادهای کلیک و اسکرول.
- **تست و مستندسازی:** مشارکت در چرخه دیباگ و بررسی عملکرد شبکه در شرایط واقعی، و همچنین نگارش گزارش نهایی مربوط به بخش ارتباطات و شبکه سمت اندروید (به صورت مشترک با ارسلان).

سید نوید هاشمی - 810101549

نقش اصلی: توسعه‌دهنده سمت سرور (Back-end)، مهندس یکپارچه‌سازی و مستندسازی بک‌اند

- **توسعه سرور پایتون:** پیاده‌سازی کامل اسکریپت سمت رایانه، مدیریت سوکت UDP، دیکد بسته‌های JSON و تبدیل آن‌ها به حرکات دقیق ماوس با کتابخانه PyAutoGUI.
- **دیباگ یکپارچه و بهینه‌سازی حرکتی:** تست مداوم ارتباط بین گوشی و لپ‌تاپ؛ شناسایی باگ‌های حرکتی و اعمال تغییرات ریاضیاتی در کدهای `MainActivity.java` و `NetworkClient.java`.
- **مستندسازی و ارائه نهایی:** نگارش کامل گزارش مربوط به بخش سرور (Python)، و همچنین ضبط ویدیوی نهایی ارائه پروژه.

بخش سمت اندروید

بررسی اجمالی

در این بخش، پیاده‌سازی سمت اندروید پروژه بررسی می‌شود. این بخش از برنامه وظیفه دارد داده‌های خام سنسورهای گوشی را دریافت کند، آن‌ها را پردازش و کالبره کند، حرکت کاربر را به فرمان‌های قابل فهم برای ماوس تبدیل کند و در نهایت این فرمان‌ها را برای سیستم مقصد ارسال نماید. به طور کلی، منطق اندروید پروژه را می‌توان به دو قسمت اصلی تقسیم کرد: بخش کدنویسی و پردازش برنامه، و بخش رابط کاربری اپلیکیشن.

در بخش کدنویسی، دو فایل اصلی نقش دارند: `MainActivity.java` و `NetworkClient.java`. فایل `MainActivity.java` هسته اصلی برنامه است و مسئولیت‌هایی مانند راه‌اندازی سنسورها، دریافت رویدادهای سنسوری، کالیبراسیون، تشخیص حرکت، کلیک و اسکرول، و آماده‌سازی داده‌های خروجی را بر عهده دارد. در کنار آن، فایل `NetworkClient.java` وظیفه برقراری ارتباط شبکه‌ای با لپتاپ را انجام می‌دهد. این کلاس داده‌های تولیدشده توسط `MainActivity` را در قالب پیام‌های مشخص از طریق UDP ارسال می‌کند و برای فرمان‌های حساس مانند کلیک و اسکرول، دریافت تأییدیه را نیز مدیریت می‌کند.

بخش دوم مربوط به رابط کاربری اپلیکیشن است. در این قسمت، کاربر می‌تواند IP و پورت سیستم مقصد را وارد کند، فرایند ارسال داده را شروع یا متوقف کند، وارد صفحه کالیبراسیون شود و وضعیت سنسورها، عملیات کلیک، اسکرول و حرکت ماوس را مشاهده کند. همچنین رابط کاربری شامل یک نمایشگر ساده برای نشان دادن جهت حرکت تشخیص داده شده است که به کاربر کمک می‌کند عملکرد برنامه را هنگام تست و دیباگ بهتر دنبال کند.

بنابراین، در ادامه این بخش ابتدا ساختار و منطق فایل‌های `MainActivity.java` و `NetworkClient.java` بررسی می‌شود و سپس طراحی و نقش رابط کاربری اپلیکیشن توضیح داده خواهد شد.

بخش اول: کدنویسی

در بخش کدنویسی سمت اندروید، هدف اصلی این است که گوشی بتواند مانند یک کنترل‌کننده حرکتی عمل کند. برای این کار، برنامه باید داده‌های خام سنسورها را از سیستم عامل اندروید دریافت

کند، خطاهای اولیه آن‌ها را تا حد امکان کاهش دهد، حرکت گوشی را به فرمان‌های قابل استفاده برای ماوس تبدیل کند و سپس این فرمان‌ها را به لپ‌تاپ ارسال کند.

این بخش از پیاده‌سازی در دو فایل اصلی انجام شده است. فایل `MainActivity.java` بخش اصلی منطق برنامه را در بر می‌گیرد. در این فایل، سنسورها راه‌اندازی می‌شوند، داده‌های شتاب‌سنج،ژیروسکوپ و مغناطیس‌سنج دریافت و کالیبره می‌شوند، حرکت گوشی تحلیل می‌شود و فرمان‌هایی مانند جابه‌جایی مکان‌نما، کلیک و اسکرول تولید می‌شوند. همچنین بخشی از وضعیت برنامه و نتیجه پردازش‌ها از همین فایل به رابط کاربری منتقل می‌شود.

در کنار آن، فایل `NetworkClient.java` مسئول ارتباط شبکه‌ای با برنامه سمت لپ‌تاپ است. این فایل داده‌های تولیدشده در `MainActivity` را در قالب بسته‌های UDP ارسال می‌کند. از آنجا که فرمان‌هایی مثل کلیک و اسکرول نباید از بین بروند، در این فایل برای این نوع فرمان‌ها مکانیزم دریافت تأییدیه یا ACK نیز در نظر گرفته شده است.

در ادامه، ابتدا فایل `MainActivity.java` به عنوان هسته اصلی پردازش برنامه بررسی می‌شود و سپس فایل `NetworkClient.java` که وظیفه ارسال داده‌ها به لپ‌تاپ را بر عهده دارد توضیح داده خواهد شد.

فایل `Mainactivity.java`

فایل `MainActivity.java` هسته اصلی برنامه اندرویدی پروژه Air Mouse است. این فایل وظیفه دریافت داده از سنسورهای گوشی، انجام کالیبراسیون، پردازش حرکت گوشی، تشخیص فرمان‌های حرکت ماوس، کلیک و اسکرول، نمایش وضعیت روی رابط کاربری و ارسال داده‌های نهایی به بخش شبکه را بر عهده دارد.

این کلاس از `Activity` ارث‌بری کرده و رابط `SensorEventListener` را پیاده‌سازی می‌کند. به همین دلیل، می‌تواند به صورت مستقیم رویدادهای سنسوری را از سیستم‌عامل اندروید دریافت کند و با هر تغییر در داده‌های سنسور، پردازش لازم را انجام دهد.

```
public class MainActivity extends Activity implements SensorEventListener {  
  
    private static final int DEFAULT_PORT = 5000;  
  
    private static final int GYROSCOPE_CALIBRATION_SAMPLES = 100;  
    private static final int MAGNETOMETER_CALIBRATION_SAMPLES = 1000;  
    private static final int ACCEL_SAMPLES_PER_STEP = 100;  
    private static final float GRAVITY_EARTH = 9.81f;  

```

1. تعریف ثابت‌ها و متغیرهای اصلی

در ابتدای فایل، تعدادی ثابت برای کنترل رفتار برنامه تعریف شده‌اند. این ثابت‌ها شامل تعداد نمونه‌های لازم برای کالیبراسیون، آستانه‌های تشخیص کلیک و اسکرول، زمان‌های cooldown و ضرایب حساسیت حرکت ماوس هستند.

برای مثال:

```
private static final float CLICK_GYROSCOPE_THRESHOLD = 2.5f;
```

```
private static final long CLICK_TIMEOUT_MS = 600;
```

```
private static final float SCROLL_ACCEL_THRESHOLD = 1.8f;
```

```
private static final double GAIN_X = 3500.0;
```

```
private static final double GAIN_Y = 3500.0;
```

ثابت `CLICK_GYROSCOPE_THRESHOLD` مشخص می‌کند که چرخش گوشی حول محور `Y` باید از چه مقداری بیشتر شود تا به عنوان کلیک شناخته شود. ثابت `CLICK_TIMEOUT_MS` نیز برای جلوگیری از تشخیص چندباره کلیک در یک حرکت استفاده می‌شود.

همچنین `SCROLL_ACCEL_THRESHOLD` آستانه تشخیص حرکت سریع برای اسکرول است. ضرایب `GAIN_X` و `GAIN_Y` میزان حساسیت حرکت ماوس را مشخص می‌کنند. اگر این ضرایب بزرگ‌تر شوند، با حرکت کمتر گوشی، مکان‌نما بیشتر جابه‌جا می‌شود.

در این بخش، متغیرهای مربوط به سنسورها، وضعیت کالیبراسیون، وضعیت اجرای موس، موقعیت نشانگر روی صفحه گوشی و ارتباط شبکه‌ای نیز تعریف شده‌اند.

```

private static final int DEFAULT_PORT = 5000;

private static final int GYROSCOPE_CALIBRATION_SAMPLES = 100;
private static final int MAGNETOMETER_CALIBRATION_SAMPLES = 1000;
private static final int ACCEL_SAMPLES_PER_STEP = 100;
private static final float GRAVITY_EARTH = 9.81f;

// Gesture detection thresholds and timing values
private static final float CLICK_GYROSCOPE_THRESHOLD = 2.5f;
private static final long CLICK_TIMEOUT_MS = 600;
private static final long SCROLL_COOLDOWN_MS = 80;

// Acceleration thresholds used to detect push/pull scrolling gestures
private static final float SCROLL_ACCEL_THRESHOLD = 1.8f;
private static final float PUSH_PULL_THRESHOLD = 1.5f;
private static final float ALPHA_GRAVITY = 0.8f;

// Mouse movement sensitivity gains
private static final double GAIN_X = 3500.0;
private static final double GAIN_Y = 3500.0;

```

2. چرخه حیات برنامه

در تابع `onCreate`، اجزای اصلی برنامه آماده‌سازی می‌شوند:

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    bindViews();
    setupSensors();
    setupUiActions();
    checkAvailableSensors();
    resetActionIndicator();
}

```

تابع `bindViews` اجزای رابط کاربری را به متغیرهای جاوا متصل می‌کند. این اجزا شامل دکمه‌ها، فیلد ورود IP و پورت، `TextView`های نمایش مقدار سنسورها و وضعیت عملیات هستند.

تابع `setupSensors` سرویس سنسور اندروید را دریافت کرده و سنسورهای مورد نیاز برنامه را مقداردهی می‌کند. تابع `setupUiActions` نیز مشخص می‌کند که هر دکمه در رابط کاربری چه عملی انجام دهد.

در تابع `onResume` سنسورها ثبت می‌شوند تا برنامه شروع به دریافت داده کند:


```
sensorManager.registerListener(this, sensor,  
SensorManager.SENSOR_DELAY_GAME);
```

استفاده از `SENSOR_DELAY_GAME` باعث می‌شود داده‌های سنسور با نرخ نسبتاً مناسبی دریافت شوند و حرکت ماوس نرم‌تر باشد.

در تابع `onPause` نیز سنسورها غیرفعال می‌شوند و در صورت فعال بودن ارتباط شبکه، متوقف می‌شود. این کار از مصرف بی‌دلیل منابع گوشی جلوگیری می‌کند.

```
// Android lifecycle  
@Override  
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
  
    bindViews();  
    setupSensors();  
    setupUiActions();  
    checkAvailableSensors();  
    resetActionIndicator();  
}  
  
@Override  
protected void onResume() {  
    super.onResume();  
    registerAvailableSensors();  
}  
  
@Override  
protected void onPause() {  
    super.onPause();  
    if (sensorManager != null) {  
        sensorManager.unregisterListener(this);  
    }  
    stopMouseClient();  
}
```

3. راه‌اندازی سنسورها

در این برنامه از سه سنسور خام استفاده شده است:

`Sensor.TYPE_ACCELEROMETER`

Sensor.TYPE_GYROSCOPE

Sensor.TYPE_MAGNETIC_FIELD

سنسور ژيروسکوپ برای تشخیص چرخش گوهی استفاده می‌شود. محورهای X و Z ژيروسکوپ برای تولید حرکت عمودی و افقی ماوس استفاده شده‌اند و محور Y برای تشخیص کلیک به کار رفته است.

شتاب‌سنج برای تشخیص حرکت سریع گوهی در راستای محور Y استفاده شده است. از این حرکت برای تشخیص اسکرول کمک گرفته می‌شود.

مغناطیس‌سنج نیز در برنامه خوانده و کالیبره می‌شود تا استفاده از سنسورهای خام طبق صورت پروژه پوشش داده شود.

در تابع `setupSensors` ابتدا `SensorManager` دریافت می‌شود و سپس سنسورهای موردنیاز از سیستم گرفته می‌شوند:

```
accelerometer =
```

```
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
```

```
gyroscope = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
```

```
magnetometer =
```

```
sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD);
```

بعد از آن، وجود یا عدم وجود هر سنسور بررسی می‌شود. اگر سنسورهای ضروری مثل شتاب‌سنج یا ژيروسکوپ موجود نباشند، برنامه اجازه شروع ماوس را نمی‌دهد.

```

private void setupSensors() {
    sensorManager = (SensorManager) getSystemService(SENSOR_SERVICE);

    if (sensorManager == null) {
        setCalibrationStatus("Status: Sensor service unavailable", R.color.error_red);
        disableAllSensorButtons();
        return;
    }

    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
    gyroscope = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
    magnetometer = sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD);

    hasAccelerometer = accelerometer != null;
    hasGyroscope = gyroscope != null;
    hasMagnetometer = magnetometer != null;
}

```

4. دریافت رویدادهای سنسور

تابع `onSensorChanged` هر بار که یکی از سنسورها داده جدیدی تولید کند اجرا می‌شود. در این تابع ابتدا نوع سنسور بررسی می‌شود و سپس داده به تابع مخصوص همان سنسور ارسال می‌شود:

```

@Override
public void onSensorChanged(SensorEvent event) {
    float[] sensorValues = event.values.clone();
    int sensorType = event.sensor.getType();

    if (sensorType == Sensor.TYPE_ACCELEROMETER) {
        Trace.beginSection("AirMouse_Accel");
        handleAccelerometer(sensorValues);
        Trace.endSection();
    } else if (sensorType == Sensor.TYPE_MAGNETIC_FIELD) {
        Trace.beginSection("AirMouse_Magneto");
        handleMagnetometer(sensorValues);
        Trace.endSection();
    } else if (sensorType == Sensor.TYPE_GYROSCOPE) {
        Trace.beginSection("AirMouse_Gyro");
        handleGyroscope(sensorValues, event.timestamp);
        Trace.endSection();
    }
}

```

در اطراف پردازش هر سنسور از `Trace.beginSection` و `Trace.endSection` استفاده شده است. این کار برای تحلیل عملکرد برنامه با ابزار Perfetto مفید است، چون می‌توان زمان اجرای هر بخش را در `trace` مشاهده کرد.

5. کالیبراسیون ژيروسکوپ

ژيروسکوپ حتی زمانی که گوشی ثابت است ممکن است مقدار غیرصفر تولید کند. این خطا معمولاً به دلیل `bias` سنسور است. برای کاهش این خطا، در برنامه یک مرحله کالیبراسیون ژيروسکوپ در نظر گرفته شده است.

در تابع `startGyroscopeCalibration`، وضعیت کالیبراسیون فعال می‌شود و لیست نمونه‌ها پاک می‌شود:

```
private void startGyroscopeCalibration() {  
    resetAllCalibrationStates();  
    gyroscopeSamples.clear();  
    isGyroscopeCalibrating = true;  
    setCalibrationStatus("Status: Calibrating Gyroscope... Keep phone still", R.color.warning_orange);  
}
```

در زمان کالیبراسیون، کاربر باید گوشی را ثابت نگه دارد. سپس در تابع `handleGyroscope` تعداد مشخصی نمونه از خروجی ژيروسکوپ جمع‌آوری می‌شود:

```
private static final int GYROSCOPE_CALIBRATION_SAMPLES = 100;
```

پس از جمع‌آوری نمونه‌ها، میانگین هر محور محاسبه شده و به عنوان `bias` ذخیره می‌شود:

```
gyroscopeBias[0] = sum[0] / gyroscopeSamples.size();
```

```
gyroscopeBias[1] = sum[1] / gyroscopeSamples.size();
```

```
gyroscopeBias[2] = sum[2] / gyroscopeSamples.size();
```

در ادامه، هنگام استفاده عادی از ژيروسکوپ، مقدار `bias` از داده خام کم می‌شود:

```
calibratedGyroscope[0] = sensorValues[0] - gyroscopeBias[0];
```

این کار باعث می‌شود وقتی گوشی ثابت است، مقدار خروجی ژيروسکوپ به صفر نزدیک‌تر شود و ماوس در حالت سکون حرکت ناخواسته نداشته باشد.

در شکل زیر قطعه کد مربوط به کالیبریشن در تابع هندل ژيروسکوپ را مشاهده می کنید.

```
private void handleGyroscope(float[] sensorValues, long timestamp) {
    hasGyroscopeData = true;

    if (isGyroscopeCalibrating) {
        gyroscopeSamples.add(sensorValues.clone());
        if (gyroscopeSamples.size() >= GYROSCOPE_CALIBRATION_SAMPLES) {
            isGyroscopeCalibrating = false;
            float[] sum = new float[3];
            for (float[] sample : gyroscopeSamples) {
                sum[0] += sample[0]; sum[1] += sample[1]; sum[2] += sample[2];
            }
            gyroscopeBias[0] = sum[0] / gyroscopeSamples.size();
            gyroscopeBias[1] = sum[1] / gyroscopeSamples.size();
            gyroscopeBias[2] = sum[2] / gyroscopeSamples.size();
            setCalibrationStatus("Status: Gyroscope Calibrated!", R.color.success_green);
        }
        return;
    }
}
```

6. کالیبراسیون شتابسنج

برای کالیبراسیون شتابسنج از روش شش وضعیتی استفاده شده است. در این روش گوشی در شش جهت مختلف قرار داده می شود و در هر مرحله تعدادی نمونه از سنسور گرفته می شود.

در برنامه، تعداد نمونه های هر مرحله با ثابت زیر مشخص شده است:

```
private static final int ACCEL_SAMPLES_PER_STEP = 100;
```

در هر مرحله، مجموع داده های سه محور ذخیره می شود و پس از رسیدن تعداد نمونه ها به مقدار تعیین شده، میانگین آن مرحله محاسبه می شود.

بعد از پایان شش مرحله، تابع `finishAccelerometerCalibration` اجرا می شود. در این تابع، برای هر محور کمینه و بیشینه مقادیر ثبت شده محاسبه می شود. سپس `offset` و `scale` به دست می آیند:

```
accelerometerOffset[axis] = (max + min) / 2.0f;
```

```
accelerometerScale[axis] = (max - min) / (2.0f * GRAVITY_EARTH);
```

شکل زیر کد مربوط به این تابع را نشان می دهد.

```
private void finishAccelerometerCalibration() {
    for (int axis = 0; axis < 3; axis++) {
        float min = Float.MAX_VALUE;
        float max = -Float.MAX_VALUE;
        for (int step = 0; step < 6; step++) {
            if (accelStepAverages[step][axis] < min) min = accelStepAverages[step][axis];
            if (accelStepAverages[step][axis] > max) max = accelStepAverages[step][axis];
        }
        accelerometerOffset[axis] = (max + min) / 2.0f;
        accelerometerScale[axis] = (max - min) / (2.0f * GRAVITY_EARTH);
        if (Math.abs(accelerometerScale[axis]) < 0.0001f) accelerometerScale[axis] = 1.0f;
    }
    isAccelerometerCalibrating = false;
    calibrateAccelerometerButton.setText("CALIBRATE ACCEL");
    setCalibrationStatus("Status: Accelerometer 6-Point Calibrated!", R.color.success_green);
}
```

پس از آن، داده خام شتاب‌سنج با استفاده از offset و scale اصلاح می‌شود:

```
accelerometerCurrent[0] = (sensorValues[0] - accelerometerOffset[0]) /
    accelerometerScale[0];
```

این کالیبراسیون باعث کاهش خطای offset و scale در خروجی شتاب‌سنج می‌شود.

قطعه کد زیر بخش کالیبریشن پیاده سازی شده در تابع هندل شتاب سنج است که از تابع `finishAccelerometerCalibration` استفاده می‌کند.

```

private void handleAccelerometer(float[] sensorValues) {
    hasAccelerometerData = true;

    if (isAccelerometerCalibrating) {
        if (isCollectingAccel) {
            accelStepSums[0] += sensorValues[0];
            accelStepSums[1] += sensorValues[1];
            accelStepSums[2] += sensorValues[2];
            accelCollectionCount++;

            if (accelCollectionCount >= ACCEL_SAMPLES_PER_STEP) {
                isCollectingAccel = false;
                int stepIndex = accelCalibStep - 1;
                accelStepAverages[stepIndex][0] = accelStepSums[0] / ACCEL_SAMPLES_PER_STEP;
                accelStepAverages[stepIndex][1] = accelStepSums[1] / ACCEL_SAMPLES_PER_STEP;
                accelStepAverages[stepIndex][2] = accelStepSums[2] / ACCEL_SAMPLES_PER_STEP;

                accelCalibStep++;
                if (accelCalibStep > 6) {
                    finishAccelerometerCalibration();
                } else {
                    calibrateAccelerometerButton.setText("RECORD " + accelCalibStep + "/6");
                    String[] instructions = {
                        "Place screen UP & press RECORD",
                        "Place screen DOWN & press RECORD",
                        "Stand phone VERTICALLY & press RECORD",
                        "Hold phone UPSIDE DOWN & press RECORD",
                        "Rest on RIGHT EDGE & press RECORD",
                        "Rest on LEFT EDGE & press RECORD"
                    };
                    setCalibrationStatus("Step " + accelCalibStep + "/6: " + instructions[accelCalibStep-1], R.color.warning_orange);
                }
            }
        }
    }
    return;
}

```

7. کالیبراسیون مغناطیس سنج

برای کالیبراسیون مغناطیس سنج، کاربر باید گوشی را به شکل عدد ۸ انگلیسی در فضا حرکت دهد. در این حالت، برنامه مقدار حداقل و حداکثر هر محور را ذخیره می‌کند:

`magnetometerMin[i]`

`magnetometerMax[i]`

پس از جمع‌آوری تعداد کافی نمونه، offset و scale هر محور محاسبه می‌شود:

`magnetometerOffset[i] = (magnetometerMax[i] + magnetometerMin[i]) / 2.0f;`

`magnetometerScale[i] = (magnetometerMax[i] - magnetometerMin[i]) / 2.0f;`

سپس مقدار تصحیح‌شده هر محور از رابطه زیر به دست می‌آید:

`corrected = (raw - offset) / scale`

این روش به کاهش خطاهای ناشی از offset و تغییر مقیاس در مغناطیس‌سنج کمک می‌کند. کد تابع `startMagnetometerCalibration` تقریباً شبیه به شروع کالیبره کردن ژيروسکوپ است اما بخش کالیبریشن آن در تابع هندل مغناطیس‌سنج را در شکل زیر نمایش می‌دهیم.

```
private void handleMagnetometer(float[] sensorValues) {
    hasMagnetometerData = true;

    if (isMagnetometerCalibrating) {
        magnetometerSampleCount++;
        for (int i = 0; i < 3; i++) {
            if (sensorValues[i] < magnetometerMin[i]) magnetometerMin[i] = sensorValues[i];
            if (sensorValues[i] > magnetometerMax[i]) magnetometerMax[i] = sensorValues[i];
        }

        if (magnetometerSampleCount >= MAGNETOMETER_CALIBRATION_SAMPLES) {
            isMagnetometerCalibrating = false;
            for (int i = 0; i < 3; i++) {
                magnetometerOffset[i] = (magnetometerMax[i] + magnetometerMin[i]) / 2.0f;
                magnetometerScale[i] = (magnetometerMax[i] - magnetometerMin[i]) / 2.0f;
                if (Math.abs(magnetometerScale[i]) < 0.0001f) magnetometerScale[i] = 1.0f;
            }
            setCalibrationStatus("Status: Magnetometer Calibrated!", R.color.success_green);
        }
        return;
    }
}
```

8. پردازش اصلی حرکت در تابع `processMotion`

تابع `processMotion` مهم‌ترین بخش فایل `MainActivity.java` است. این تابع داده‌های ژيروسکوپ و شتاب‌سنج را به فرمان‌های حرکت ماوس، کلیک و اسکرول تبدیل می‌کند.

ابتدا اختلاف زمانی بین دو نمونه متوالی ژيروسکوپ محاسبه می‌شود:

```
float deltaTime = (timestamp - lastTimestamp) / 1_000_000_000.0f;
```

چون timestamp سنسور بر حسب نانوثانیه است، با تقسیم 1000000000 به ثانیه تبدیل می‌شود. این مقدار برای محاسبه جابه‌جایی ماوس استفاده می‌شود.

اگر مقدار `deltaTime` غیرمنطقی باشد، تابع بدون پردازش بیشتر خارج می‌شود:

```
if (deltaTime <= 0.0f || deltaTime > 1.0f) return;
```

این شرط از پرش ناگهانی ماوس در شرایطی مثل توقف موقت برنامه یا تأخیر زیاد جلوگیری می‌کند.


```
private void processMotion(float[] gyroscopeValues, long timestamp) {
    if (!hasAccelerometerData) return;

    if (lastTimestamp == 0) {
        lastTimestamp = timestamp;
        return;
    }

    float deltaTime = (timestamp - lastTimestamp) / 1_000_000_000.0f;
    lastTimestamp = timestamp;

    if (deltaTime <= 0.0f || deltaTime > 1.0f) return;
}
```

9. تخمین gravity و استخراج شتاب خطی

برای تشخیص اسکرول، لازم است شتاب خطی گوشی از شتاب گرانش جدا شود. برای این کار از یک فیلتر پایین‌گذر ساده استفاده شده است:

```
gravity[0] = ALPHA_GRAVITY * gravity[0] + (1.0f - ALPHA_GRAVITY) *
accelerometerCurrent[0];
```

این فیلتر مقدار تقریبی gravity را از داده شتاب‌سنج استخراج می‌کند. سپس شتاب خطی محور Y با کم کردن gravity از داده شتاب‌سنج به دست می‌آید:

```
float linearAccelerationY = accelerometerCurrent[1] - gravity[1];
```

این مقدار برای تشخیص حرکت سریع گوشی در راستای محور Y استفاده می‌شود.

نکته مهم این است که در این برنامه از فیلترهای پیچیده‌ای مثل Kalman، Mahony یا Madgwick استفاده نشده است. روش استفاده‌شده در این بخش یک low-pass filter ساده برای تخمین gravity است.

```
// Low-pass filter for estimating gravity from accelerometer data
gravity[0] = ALPHA_GRAVITY * gravity[0] + (1.0f - ALPHA_GRAVITY) * accelerometerCurrent[0];
gravity[1] = ALPHA_GRAVITY * gravity[1] + (1.0f - ALPHA_GRAVITY) * accelerometerCurrent[1];
gravity[2] = ALPHA_GRAVITY * gravity[2] + (1.0f - ALPHA_GRAVITY) * accelerometerCurrent[2];

float linearAccelerationY = accelerometerCurrent[1] - gravity[1];
long currentTime = System.currentTimeMillis();
```

10. تشخیص کلیک

طبق صورت پروژه، کلیک باید با حرکت یا چرخش سریع گوشی حول محور Y تشخیص داده شود. در این کد، اگر مقدار محور Y ژيروسکوپ از آستانه منفی مشخصی کمتر شود، برنامه آن را به عنوان کلیک در نظر می‌گیرد:

```
if (gyroscopeValues[1] < -CLICK_GYROSCOPE_THRESHOLD) {  
  
    clickEvent = 1;  
  
}
```

برای جلوگیری از ثبت چند کلیک پشت سر هم در اثر یک حرکت، از متغیر `lastClickTime` و مقدار `CLICK_TIMEOUT_MS` استفاده شده است:

```
lastClickTime = currentTime + CLICK_TIMEOUT_MS;
```

همچنین پس از تشخیص کلیک، حرکت موس برای مدت کوتاهی قفل می‌شود:

```
gestureLockoutTime = currentTime + 300;
```

این کار به این دلیل انجام شده که هنگام حرکت سریع گوشی برای کلیک، ممکن است دست کمی جابه‌جا شود و این جابه‌جایی به اشتباه به عنوان حرکت ماوس تفسیر شود.

در انتهای تابع نیز اگر بسته شامل کلیک باشد، مقدار حرکت صفر می‌شود:

```
if (clickEvent == 1) {  
  
    deltaX = 0.0;  
  
    deltaY = 0.0;  
  
}
```

این بخش باعث می‌شود کلیک همراه با جابه‌جایی ناخواسته ارسال نشود و کلیک در نقطه فعلی مکان‌نما انجام شود.

```
// Detect left-click using fast rotation around the Y axis
if (currentTime > lastClickTime) {
    if (gyroscopeValues[1] < -CLICK_GYROSCOPE_THRESHOLD) {
        clickEvent = 1;
        lastClickTime = currentTime + CLICK_TIMEOUT_MS;
        gestureLockoutTime = currentTime + 300; // قفل کردن حرکت موس حین کلیک
        triggerVisualAction("CLICK!", R.color.error_red, 600);
    }
}
```

```
// A click packet must not contain cursor movement
if (clickEvent == 1) {
    deltaX = 0.0;
    deltaY = 0.0;
}
```

11. تشخیص اسکرول

برای تشخیص اسکرول از شتاب خطی محور Y استفاده شده است. اگر مقدار `linearAccelerationY` از آستانه مثبت بیشتر شود، یک جهت اسکرول در نظر گرفته می‌شود و اگر از آستانه منفی کمتر شود، جهت مخالف اسکرول تشخیص داده می‌شود:

```
if (linearAccelerationY > SCROLL_ACCEL_THRESHOLD) currentDirection = 1;
```

```
else if (linearAccelerationY < -SCROLL_ACCEL_THRESHOLD) currentDirection = -1;
```

برای جلوگیری از تشخیص چندباره اسکرول در مدت کوتاه، از `cooldown` استفاده شده است:

```
scrollCooldownTime = currentTime + SCROLL_COOLDOWN_MS;
```

همچنین وقتی کاربر گوشی را برای اسکرول حرکت می‌دهد، برگشت دست به حالت اولیه ممکن است به اشتباه به عنوان اسکرول در جهت مخالف تشخیص داده شود. برای جلوگیری از این مشکل از متغیرهای زیر استفاده شده است:

`lastScrollDirection`

`oppositeDirectionLockTime`

`OPPOSITE_DIRECTION_TIMEOUT_MS`

این بخش باعث می‌شود اسکرول پایدارتر باشد و حرکت برگشتی دست باعث ایجاد فرمان اشتباه نشود.

```
// Detect scroll gestures using strong push/pull motion on the Y axis
if (currentTime > scrollCooldownTime && !isPitching) {
    int currentDirection = 0;
    if (linearAccelerationY > SCROLL_ACCEL_THRESHOLD) currentDirection = 1;
    else if (linearAccelerationY < -SCROLL_ACCEL_THRESHOLD) currentDirection = -1;

    if (currentDirection != 0 && (currentDirection == lastScrollDirection || currentTime > oppositeDirectionLockTime)) {
        scrollEvent = currentDirection;
        lastScrollDirection = currentDirection;
        oppositeDirectionLockTime = currentTime + OPPOSITE_DIRECTION_TIMEOUT_MS;
        scrollCooldownTime = currentTime + SCROLL_COOLDOWN_MS;

        // Lock mouse while scrolling
        gestureLockoutTime = currentTime + 400;
        triggerVisualAction(scrollEvent == 1 ? "SCROLL UP" : "SCROLL DOWN", R.color.primary_blue, 80);
    }
}
```

12. محاسبه حرکت مکان‌نما

حرکت ماوس از داده‌هایژیروسکوپ محاسبه می‌شود. محور **Z**ژیروسکوپ برای حرکت افقی و محور **X** برای حرکت عمودی مکان‌نما استفاده شده است:

```
float rateX = gyroscopeValues[0];
```

```
float rateZ = gyroscopeValues[2];
```

سپس این نرخ‌های چرخش با توجه به **deltaTime** و ضرایب حساسیت به جابه‌جایی مکان‌نما تبدیل می‌شوند:

```
deltaX = -rateZ * GAIN_X * deltaTime;
```

```
deltaY = -rateX * GAIN_Y * deltaTime;
```

در اینجا به جای ارسال موقعیت مطلق، مقدار جابه‌جایی نسبت به لحظه قبل ارسال می‌شود. این موضوع با صورت پروژه سازگار است، چون داده ارسالی نباید مقدار مطلق مکان‌نما باشد، بلکه باید تغییرات حرکت را نشان دهد.

برای جلوگیری از حرکت ناخواسته ماوس در اثر نویز سنسور یا لرزش طبیعی دست، از **deadzone** استفاده شده است:

```
final float DEADZONE = 0.06f;
```

اگر مقدار چرخش کمتر از این آستانه باشد، صفر در نظر گرفته می‌شود:

```
if (Math.abs(rateX) < DEADZONE) rateX = 0.0f;
```

```
if (Math.abs(rateZ) < DEADZONE) rateZ = 0.0f;
```

این بخش نقش مهمی در نرم‌تر شدن حرکت موس و جلوگیری از لرزش مکان‌نما دارد.

```
// Calculate cursor movement only when no gesture lockout is active
if (currentTime > gestureLockoutTime && !isPushPullGesture) {
    float rateX = gyroscopeValues[0]; // حرکت عمودی
    float rateZ = gyroscopeValues[2]; // حرکت افقی

    // Deadzone removes small hand tremors when the phone is almost still
    final float DEADZONE = 0.06f;
    if (Math.abs(rateX) < DEADZONE) rateX = 0.0f;
    if (Math.abs(rateZ) < DEADZONE) rateZ = 0.0f;

    deltaX = -rateZ * GAIN_X * deltaTime;
    deltaY = -rateX * GAIN_Y * deltaTime;
}
```

13. نمایش وضعیت روی رابط کاربری

در کنار ارسال داده به لپ‌تاپ، برنامه وضعیت حرکت و فرمان‌ها را روی رابط کاربری گوشی نیز نمایش می‌دهد.

تابع `updateVisualPointer` موقعیت دایره کوچک نمایش‌داده‌شده روی صفحه را بر اساس `deltaX` و `deltaY` تغییر می‌دهد:

```
visualPointerView.setTranslationX(currentVisualX);
```

```
visualPointerView.setTranslationY(currentVisualY);
```

این بخش برای تست و دیباگ مفید است، چون کاربر می‌تواند ببیند برنامه حرکت گوشی را در چه جهتی تشخیص داده است.

```
// Update the small visual pointer shown inside the Android UI.
private void updateVisualPointer(float deltaX, float deltaY) {
    uiHandler.post(() -> {
        View parentView = (View) visualPointerView.getParent();
        if (parentView == null) return;

        float maxX = (parentView.getWidth() - visualPointerView.getWidth()) / 2.0f;
        float maxY = (parentView.getHeight() - visualPointerView.getHeight()) / 2.0f;

        currentVisualX += deltaX;
        currentVisualY += deltaY;

        if (currentVisualX > maxX) currentVisualX = maxX;
        if (currentVisualX < -maxX) currentVisualX = -maxX;
        if (currentVisualY > maxY) currentVisualY = maxY;
        if (currentVisualY < -maxY) currentVisualY = -maxY;

        visualPointerView.setTranslationX(currentVisualX);
        visualPointerView.setTranslationY(currentVisualY);
    });
}
```

تابع `triggerVisualAction` نیز برای نمایش موقت عملیات‌هایی مانند کلیک و اسکرول استفاده می‌شود:

```
triggerVisualAction("CLICK!", R.color.error_red, 600);
```

همچنین توابع `setAction` و `setCalibrationStatus` برای به‌روزرسانی متن و رنگ وضعیت برنامه و کالیبراسیون استفاده شده‌اند.

```
// Show a temporary click or scroll action on the UI.
private void triggerVisualAction(String actionLabel, int colorResId, long durationMs) {
    uiHandler.post(() -> {
        setAction("Action: " + actionLabel, colorResId);
        visualPointerView.setBackgroundResource(R.drawable.bg_pointer_dot);
        uiHandler.removeCallbacks(resetVisualRunnable);
        uiHandler.postDelayed(resetVisualRunnable, durationMs);
    });
}
```

14. نقش `gestureLockoutTime` در کاهش خطا

یکی از مشکلات مهم در این پروژه این است که هنگام انجام کلیک یا اسکرول، گوشی به صورت ناخواسته کمی در جهت‌های دیگر هم حرکت می‌کند. اگر این حرکات مستقیماً به ماوس منتقل شوند، مکان‌نما قبل از کلیک یا هنگام اسکرول جابه‌جا می‌شود.

برای کاهش این مشکل، از متغیر `gestureLockoutTime` استفاده شده است. وقتی کلیک یا اسکرول تشخیص داده می‌شود، این متغیر برای چند صد میلی‌ثانیه جلوتر تنظیم می‌شود. تا زمانی که این زمان تمام نشده باشد، برنامه حرکت عادی ماوس را محاسبه نمی‌کند:

```
if (currentTime > gestureLockoutTime && !isPushPullGesture) {  
  
    ...  
  
}
```

این کار باعث می‌شود حرکت‌های ناخواسته‌ای که همراه با `gesture` رخ می‌دهند، کمتر روی مکان‌نما اثر بگذارند.

در مجموع، فایل `MainActivity.java` بخش اصلی پردازش سمت اندروید پروژه را پیاده‌سازی می‌کند. این فایل ابتدا سنسورهای خام گوشی شامل شتاب‌سنج،ژیروسکوپ و مغناطیس‌سنج را راه‌اندازی می‌کند. سپس برای کاهش خطا، هر سنسور به روش مناسب کالیبره می‌شود. ژیروسکوپ با میانگین‌گیری در حالت سکون `bias` خود را حذف می‌کند، شتاب‌سنج با روش شش‌وضعیتی `scale` و `offset` خود را اصلاح می‌کند و مغناطیس‌سنج با روش `min/max` کالیبره می‌شود.

پس از دریافت داده‌های سنسوری، برنامه در تابع `processMotion` حرکت گوشی را تحلیل می‌کند. حرکت مکان‌نما با استفاده از محورهای `X` و `Z` ژیروسکوپ تولید می‌شود. کلیک با چرخش سریع حول محور `Y` ژیروسکوپ و اسکرول با حرکت سریع در راستای محور `Y` شتاب‌سنج تشخیص داده می‌شود. برای افزایش پایداری برنامه از روش‌هایی مانند `deadzone`، `cooldown`، `gesture` و `lockout` و فیلتر پایین‌گذر ساده برای تخمین `gravity` استفاده شده است.

در نهایت، داده‌های پردازش‌شده شامل `deltaX`، `deltaY` وضعیت کلیک و وضعیت اسکرول به کلاس `NetworkClient` ارسال می‌شوند تا به لپ‌تاپ منتقل شوند. همچنین وضعیت حرکت و عملیات‌ها روی رابط کاربری گوشی نمایش داده می‌شود تا تست و دیباگ برنامه ساده‌تر باشد.

این پیاده‌سازی تلاش می‌کند الزامات اصلی صورت پروژه را پوشش دهد: استفاده از سنسورهای خام، انجام کالیبراسیون، تولید حرکت نسبی برای مکان‌نما، تشخیص کلیک و اسکرول، جلوگیری از حرکت ناخواسته و نمایش وضعیت برنامه روی رابط کاربری.

کلاس مدیریت شبکه به عنوان پل ارتباطی اصلی میان دستگاه اندرویدی و رایانه میزبان در سیستم سایبر-فیزیکی ماوس هوایی عمل می‌کند. وظیفه محوری این ماژول، انتقال بلادرنگ داده‌های استخراج‌شده از حسگرها و فرامین حرکتی بر بستر پروتکل سریع انتقال داده است تا کمترین میزان تأخیر در جابه‌جایی نشانگر احساس شود. برای جلوگیری از تداخل عملیات شبکه‌ای با پردازش‌های فرکانس بالای حسگرها و همچنین حفظ یکپارچگی و سرعت رابط کاربری، تمامی فرآیندهای ایجاد اتصال، ارسال بسته‌ها و مانیتورینگ در یک بستر پردازشی کاملاً ایزوله و پس‌زمینه با بهره‌گیری از ابزارهای استاندارد مدیریت ریسمان پیاده‌سازی شده‌اند.

فرآیند آماده‌سازی اطلاعات در این بخش مبتنی بر ساختار سبک و استاندارد توسعه یافته است تا داده‌های اعشاری مربوط به جابه‌جایی محورها به همراه رویدادهای گسسته‌ای نظیر کلیک و اسکرول به شکلی یکپارچه جمع و بسته‌بندی شوند. این اطلاعات ساختاریافته به صورت بی‌وقفه به سمت سرور جریان پیدا می‌کنند و انتخاب پروتکل بدون اتصال به جای جایگزین‌های نیازمند به دست‌تکانی اولیه، سربار شبکه را به حداقل ممکن رسانده است. این رویکرد اجازه می‌دهد تا سیلان پیوسته مختصات حرکتی با بالاترین نرخ فریم ممکن جریان یابد و تجربه‌ای روان و کاملاً مشابه با عملکرد یک سخت‌افزار فیزیکی ماوس برای سیستم‌عامل مقصد رقم بخورد.

با توجه به ماهیت غیرقابل‌تضمین پروتکل‌های انتقال سریع، این کلاس به یک مکانیزم ابتکاری و سفارشی برای تأیید دریافت داده‌های حیاتی مجهز شده است. در حالی که از دست رفتن چند فریم گذرا در حرکت پیوسته نشانگر قابل چشم‌پوشی بوده و با فریم‌های بعدی جبران می‌شود، رویدادهای تعاملی مانند کلیک نیازمند اجرای قطعی هستند؛ از این رو سیستم در زمان بروز این فرامین حساس، جریان ارسال داده‌های حرکتی عادی را موقتاً مسدود کرده و تا زمان دریافت تأییدیه صریح از سمت سرور در یک بازه زمانی مشخص و تکرار محدود ارسال، در یک حلقه کنترل‌شده باقی می‌ماند تا تعادلی مهندسی‌شده میان سرعت اجرای بلادرنگ و قابلیت اطمینان شبکه برقرار گردد.


```

35 // Starts the UDP socket on a background thread.
36 public void start() {
37     isRunning = true;
38     networkExecutor = Executors.newSingleThreadExecutor();
39
40     networkExecutor.execute(() -> {
41         try {
42             udpSocket = new DatagramSocket();
43             serverAddress = InetAddress.getByName(serverIpAddress);
44             Log.d("AirMouseNetwork", "UDP socket started -> " + serverIpAddress + ":" + serverPort);
45         } catch (Exception e) {
46             Log.e("AirMouseNetwork", "Socket initialization failed", e);
47         }
48     });
49 }
50
51 // Stops the network executor and closes the UDP socket.
52 public void stop() {
53     isRunning = false;
54
55     if (networkExecutor != null) {
56         networkExecutor.shutdown();
57     }
58
59     if (udpSocket != null && !udpSocket.isClosed()) {
60         udpSocket.close();
61         Log.d("AirMouseNetwork", "UDP socket closed");
62     }
63 }
64

```

کلاس مدیریت شبکه با هدف برقراری ارتباط بلادرنگ بر بستر پروتکل UDP طراحی شده است تا داده‌های حرکتی با کمترین تأخیر به سرور منتقل شوند. از آنجا که عملیات شبکه‌ای در سیستم‌عامل اندروید نباید روی ریسمان اصلی (Main Thread) اجرا شوند، از کلاس ExecutorService برای ایجاد یک بستر پردازشی پس‌زمینه و ایزوله استفاده شده است. شروع و پایان این چرخه در متدهای استارت و استوپ پیاده‌سازی شده تا چرخه حیات سوکت‌ها و ریسمان‌ها به درستی مدیریت شود. این معماری تضمین می‌کند که رابط کاربری و محاسبات سنسورها درگیر تأخیرهای احتمالی شبکه نشده و نرمی حرکت ماوس حفظ گردد.

```

65     public void sendMouseData(double deltaX, double deltaY, int clickEvent, int scrollEvent) {
66         if (!isRunning || networkExecutor == null) return;
67
68         // Drop normal movement packets while waiting for ACK from a critical click/scroll packet.
69         if (isWaitingForAck && clickEvent == 0 && scrollEvent == 0) {
70             return;
71         }
72
73         networkExecutor.execute(() -> {
74             Trace.beginSection("AirMouse_UDP_Send");
75
76             try {
77                 JSONObject payload = new JSONObject();
78                 payload.put("DeltaX", deltaX);
79                 payload.put("DeltaY", deltaY);
80                 payload.put("Click", clickEvent);
81                 payload.put("Scroll", scrollEvent);
82
83                 String message = payload.toString();
84                 byte[] buffer = message.getBytes();
85
86                 if (udpSocket != null && serverAddress != null) {
87                     DatagramPacket packet = new DatagramPacket(
88                         buffer,
89                         buffer.length,
90                         serverAddress,
91                         serverPort
92                     );
93
94                     udpSocket.send(packet);
95                     Log.d("AirMouseNetwork", "Sent: " + message);
96

```

بخش اصلی ارسال داده‌ها در بدنه متد ارسال اطلاعات قرار دارد که مقادیر جابه‌جایی محورها به همراه رویدادهای کلیک و اسکرول را پردازش می‌کند. این داده‌ها درون یک ساختار استاندارد JSON جمع می‌شوند تا سرور در سیستم‌عامل مقصد بتواند بدون وابستگی به پلتفرم، آن‌ها را رمزگشایی کند. این ساختار متنی پس از تبدیل به آرایه‌ای از بایت‌ها، درون یک دیتاگرام‌پکت قرار گرفته و مستقیماً به آدرس آی‌پی و پورت هدف ارسال می‌گردد. همچنین با استفاده از ابزار تریس، مانیتورینگ دقیقی برای ثبت زمان اجرای این عملیات درون ابزارهای پروفایلینگ پیاده‌سازی شده است تا بتوان گلوگاه‌های عملکردی سیستم سایبر-فیزیکی را ارزیابی کرد

```

97         // Wait for ACK and retry critical click/scroll packets if needed.
98         if (clickEvent != 0 || scrollEvent != 0) {
99
100             isWaitingForAck = true; // // Block normal movement packets
101             boolean ackReceived = false;
102             int retryCount = 0;
103
104             udpSocket.setSoTimeout(200);
105
106             byte[] receiveBuffer = new byte[256];
107             DatagramPacket receivePacket = new DatagramPacket(receiveBuffer, receiveBuffer.length);
108
109             while (!ackReceived && retryCount < 3) {
110                 try {
111                     udpSocket.receive(receivePacket);
112
113                     String ackMessage = new String(
114                         receivePacket.getData(),
115                         0,
116                         receivePacket.getLength()
117                     );
118
119                     if (ackMessage.contains("ACK")) {
120                         ackReceived = true;
121                         Log.d("AirMouseNetwork", "ACK received");
122                     }
123
124                     } catch (SocketTimeoutException e) {
125                         retryCount++;
126                         Log.d("AirMouseNetwork", "ACK timeout retry " + retryCount);
127
128                         if (retryCount < 3) {
129                             udpSocket.send(packet);
130                             Log.d("AirMouseNetwork", "Resending packet");
131                         }
132                     }
133                 }
134
135                 udpSocket.setSoTimeout(0);
136                 isWaitingForAck = false; // Allow normal movement packets again
137             }
138
139         } catch (Exception e) {
140             Log.e("AirMouseNetwork", "Send failed", e);
141         } finally {
142             Trace.endSection();
143         }
144     }
145 });
146 }
147 }

```

از آنجا که پروتکل UDP ذاتاً تضمینی برای رسیدن داده‌ها ارائه نمی‌دهد، یک مکانیزم تأیید دریافت (ACK) سفارشی برای بسته‌های حساس در بدنه ارسال توسعه یافته است. اگر یک رویداد حیاتی مانند کلیک یا اسکرول رخ دهد، سیستم حداکثر ۲۰۰ میلی‌ثانیه منتظر دریافت پاسخ از سرور می‌ماند و در صورت عدم دریافت تأییدیه، بسته را تا سه بار مجدداً ارسال می‌کند. همزمان برای جلوگیری از ایجاد ترافیک کاذب و پر شدن صف ارسال، با استفاده از یک متغیر کنترلی از ارسال بسته‌های عادی جابه‌جاییِ ماوس در طول این دوره انتظار جلوگیری می‌شود تا پایداری ارتباط و اجرای قطعی دستورات تضمین گردد.

بخش دوم: رابط کاربری

در این بخش، رابط کاربری اپلیکیشن Air Mouse طراحی شده است. هدف اصلی این قسمت، فراهم کردن یک محیط ساده و قابل فهم برای وارد کردن تنظیمات اتصال، شروع عملکرد موس، مشاهده وضعیت لحظه‌ای سنسورها و انجام کالیبراسیون است. این بخش بیشتر نقش واسط بین کاربر و منطق اصلی برنامه را دارد و خود پردازش حرکت یا ارسال داده‌ها در این فایل انجام نمی‌شود. فایل اصلی آن `activity_main.xml` است که توضیحات اصلی از روی آن داده می شود.

رابط کاربری با استفاده از فایل XML طراحی شده و ساختار کلی آن بر پایه‌ی RelativeLayout است. داخل این لایه، دو صفحه‌ی اصلی قرار داده شده است:

1. صفحه‌ی اصلی برنامه یا `mainScreen`
2. صفحه‌ی کالیبراسیون یا `calibrationScreen`

در ابتدای اجرای برنامه، صفحه‌ی اصلی قابل مشاهده است و صفحه‌ی کالیبراسیون مخفی نگه داشته می‌شود. این کار با استفاده از ویژگی‌های `visibility=visible` برای صفحه‌ی اصلی و `visibility=gone` برای صفحه‌ی کالیبراسیون انجام شده است. بنابراین برنامه بدون نیاز به ساخت `Activity` جداگانه، بین دو نمای مختلف جابه‌جا می‌شود.

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/bg_cream">

    <ScrollView
        android:id="@+id/mainScreen"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true"
        android:visibility="visible">

        <LinearLayout ...
        </LinearLayout>
    </ScrollView>

    <ScrollView
        android:id="@+id/calibrationScreen"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="@color/bg_cream"
        android:visibility="gone">

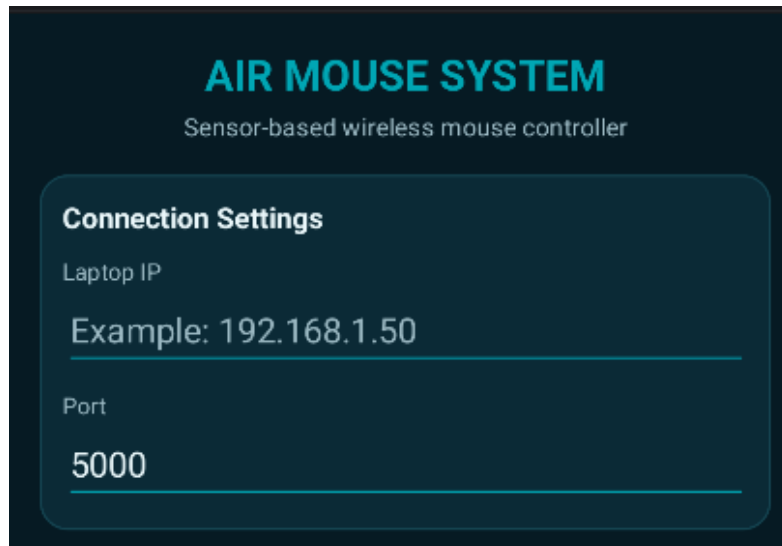
        <LinearLayout ...
        </LinearLayout>
    </ScrollView>
</RelativeLayout>

```

صفحه‌ی اصلی برنامه یا mainScreen

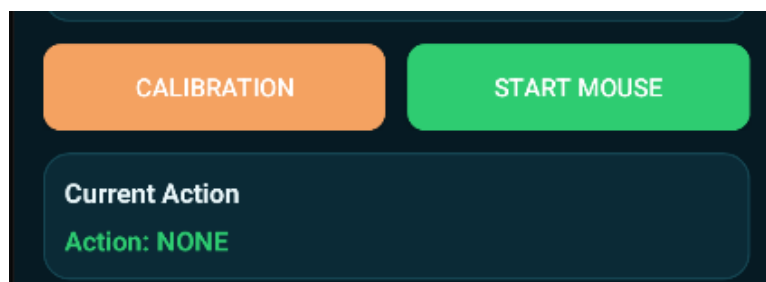
در صفحه‌ی اصلی، ابتدا عنوان برنامه با متن **AIR MOUSE SYSTEM** نمایش داده می‌شود. زیر آن نیز یک توضیح کوتاه با عنوان **Sensor-based wireless mouse controller** قرار گرفته تا مشخص شود برنامه برای کنترل موس با استفاده از سنسورهای موبایل طراحی شده است.

بخش بعدی مربوط به تنظیمات اتصال است. در این قسمت، کاربر می‌تواند آدرس IP لپ‌تاپ و شماره پورت سرور را وارد کند. برای IP از **EditText** با شناسه‌ی **ipInput** استفاده شده و برای پورت نیز **EditText** با شناسه‌ی **portInput** در نظر گرفته شده است. مقدار پیش‌فرض پورت برابر با 5000 قرار داده شده که با پورت سرور UDP هماهنگ است.



پس از تنظیمات اتصال، دو دکمه‌ی اصلی در صفحه قرار داده شده است. دکمه‌ی **CALIBRATION** با شناسه‌ی `btnOpenCalibration` برای ورود به صفحه‌ی کالیبراسیون استفاده می‌شود. دکمه‌ی **START MOUSE** با شناسه‌ی `btnStartMouse` نیز برای شروع عملکرد موس طراحی شده است. این دو دکمه در یک `LinearLayout` افقی قرار گرفته‌اند و با استفاده از `layout_weight` فضای صفحه را به صورت مساوی بین خود تقسیم می‌کنند.

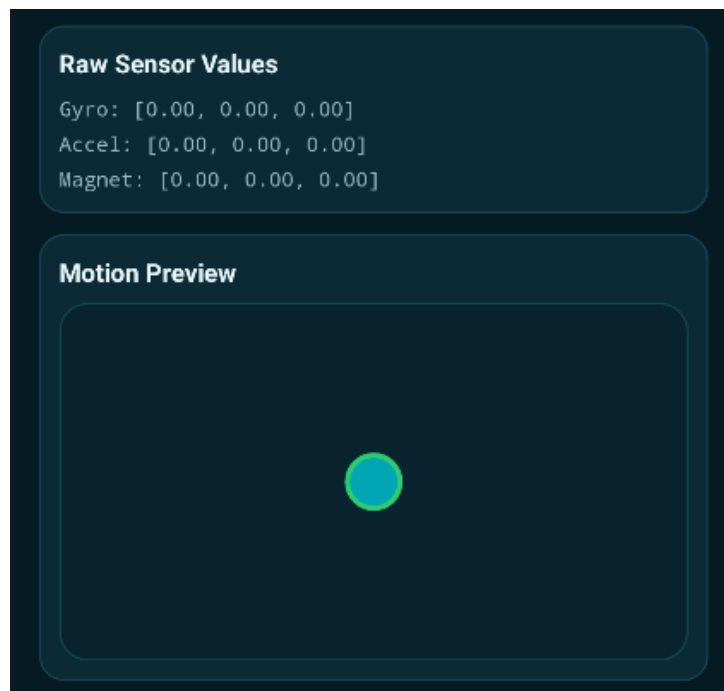
در ادامه، بخشی با عنوان `Current Action` قرار داده شده است که وضعیت فعلی عملکرد برنامه را نمایش می‌دهد. مقدار اولیه‌ی این بخش `Action: NONE` است. این قسمت برای نمایش عملیات‌هایی مانند حرکت، کلیک یا اسکرول کاربرد دارد و باعث می‌شود کاربر هنگام اجرای برنامه از وضعیت فعلی سیستم مطلع باشد.



بخش بعدی برای نمایش مقدار خام سنسورها طراحی شده است. در این قسمت سه `TextView` با شناسه‌های `txtGyro`، `txtAccel` و `txtMag` قرار دارند که به ترتیب مقادیر ژيروسکوپ، شتاب‌سنج و مغناطیس‌سنج را نمایش می‌دهند. استفاده از فونت `monospace` در این بخش باعث می‌شود

مقادیر عددی خواناتر و منظم‌تر دیده شوند. این قسمت بیشتر برای بررسی و تست عملکرد سنسورها مفید است.

در انتهای صفحه‌ی اصلی، بخش **Motion Preview** قرار دارد. این بخش شامل یک ناحیه‌ی نمایشی و یک نقطه‌ی کوچک با شناسه‌ی **visualPointer** است. این نقطه می‌تواند به عنوان پیش‌نمایش حرکت موس در داخل برنامه استفاده شود. به عبارت دیگر، کاربر می‌تواند قبل یا هنگام ارسال داده به سیستم، یک نمایش ساده از جهت حرکت را در خود اپلیکیشن ببیند.



صفحه‌ی کالیبراسیون یا calibrationScreen

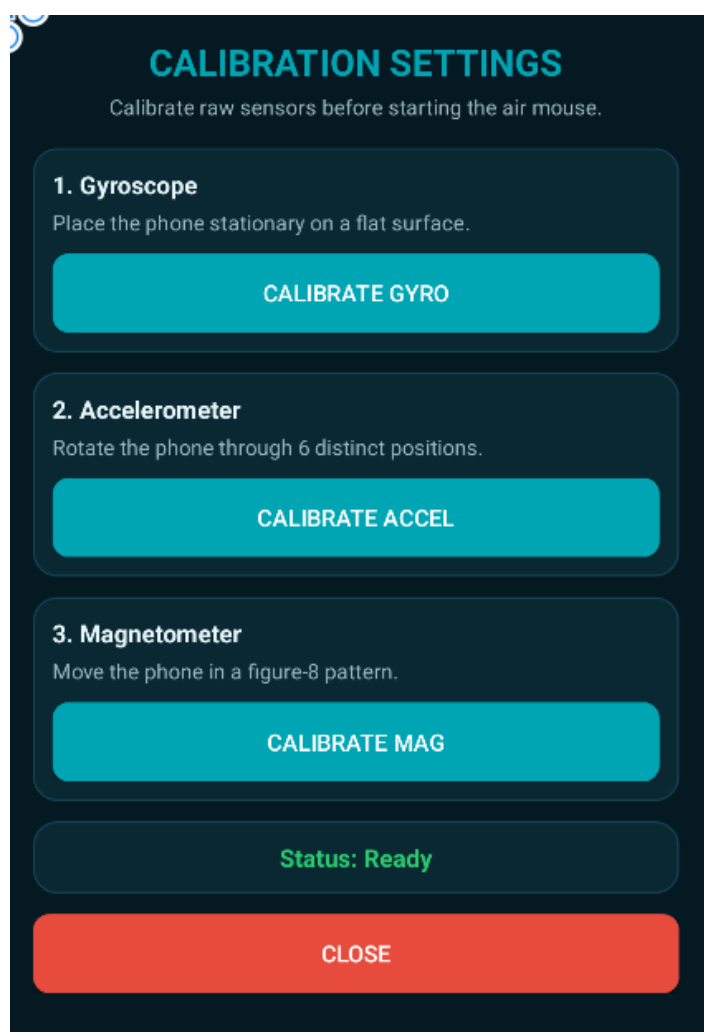
صفحه‌ی دوم رابط کاربری مربوط به کالیبراسیون است. این صفحه با عنوان **CALIBRATION** شروع می‌شود و هدف آن آماده‌سازی سنسورها قبل از شروع عملکرد موس است. در این صفحه سه بخش جداگانه برای کالیبراسیون ژيروسکوپ، شتاب‌سنج و مغناطیس‌سنج طراحی شده است.

برای ژيروسکوپ، از کاربر خواسته می‌شود گوشی را روی یک سطح صاف و ثابت قرار دهد. سپس با دکمه‌ی **CALIBRATE GYRO** عملیات کالیبراسیون انجام می‌شود. برای شتاب‌سنج، توضیح داده شده که گوشی باید در شش وضعیت مختلف چرخانده شود و دکمه‌ی **CALIBRATE ACCEL** برای

آن قرار دارد. برای مغناطیس‌سنج نیز از کاربر خواسته می‌شود گوشی را به شکل عدد ۸ حرکت دهد و دکمه‌ی **CALIBRATE MAG** برای شروع این فرآیند طراحی شده است.

در پایین صفحه‌ی کالیبراسیون، یک **TextView** با شناسه‌ی **txtCalibrationStatus** قرار دارد که وضعیت کالیبراسیون را نمایش می‌دهد. مقدار اولیه‌ی آن Status: Ready است. همچنین دکمه‌ی **CLOSE** با شناسه‌ی **btnCloseCalibration** برای بازگشت از صفحه‌ی کالیبراسیون به صفحه‌ی اصلی استفاده می‌شود.

از نظر ظاهری، در طراحی UI از رنگ‌ها و **drawable**های جداگانه استفاده شده است. برای مثال، پس‌زمینه‌ی کلی صفحه با رنگ **bg_cream** مشخص شده و کارت‌های مختلف با **bg_card** طراحی شده‌اند. دکمه‌ها نیز بر اساس کاربردهایشان ظاهر متفاوت دارند؛ مثلاً دکمه‌ی شروع با رنگ موفقیت، دکمه‌ی کالیبراسیون با رنگ هشدار و دکمه‌ی بستن با رنگ خطا نمایش داده می‌شود. این کار باعث می‌شود رابط کاربری مرتب‌تر باشد و کاربر راحت‌تر عملکرد هر دکمه را تشخیص دهد.



به طور کلی، این بخش از پروژه یک رابط کاربری ساده، سبک و کاربردی ایجاد می‌کند. کاربر می‌تواند از طریق آن تنظیمات اتصال را وارد کند، عملکرد موس را شروع کند، وضعیت سنسورها را ببیند، حرکت را به صورت ساده پیش‌نمایش بگیرد و در صورت نیاز سنسورها را کالیبره کند. این طراحی پیچیدگی غیرضروری ندارد و با نیازهای اصلی پروژه هماهنگ است.

بخش سمت رایانه شخصی

در این بخش، اسکریپت سمت سرور که به زبان پایتون توسعه یافته است، به عنوان یک واسط بین لایه شبکه و سیستم عامل رایانه عمل می‌کند. هدف اصلی این اسکریپت، دریافت بدون وقفه داده های سنسوری از طریق پروتکل UDP، رمزگشایی آن ها و تبدیلیشان به رویداد های فیزیکی ماوس در سطح سیستم عامل است. در ادامه، جزئیات پیاده سازی این برنامه در قالب بخش های مجزا تشریح شده است.

۱. **پیکربندی محیط و تنظیمات سیستم عامل:** در ابتدا از کتابخانه `pyautogui` برای شبیه سازی رفتار ماوس استفاده شده است. برای اینکه این کتابخانه بتواند در یک سناریوی بی درنگ (Real-time) و با فرکانس بالا کار کند، دو تنظیم امنیتی و پیش فرض آن تغییر یافته است:

- متغیر `pyautogui.FAILSAFE = False`: به طور پیش فرض، اگر نشانگر ماوس به گوشه های مانیتور برود، این کتابخانه یک exception پرتاب کرده و برنامه را متوقف می‌کند. از آنجا که در یک ماوس هوایی ممکن است در لحظات اولیه کالیبراسیون داده های ناگهانی تولید شود و در کل ما این محدودیت را نیاز نداریم، این ویژگی غیرفعال شده است تا سرور کرش نکند.
- متغیر `pyautogui.PAUSE = 0.0`: این کتابخانه برای طبیعی تر جلوه دادن حرکات، به صورت پیش فرض یک تاخیر ۰.۱ ثانیه ای بین هر دستور ایجاد می‌کند. ما برای داشتن یک ماوس کاملاً روان که بدون Lag کار کند، این تاخیر به صفر مطلق کاهش دادیم.

۲. **راه اندازی سوکت شبکه:** ارتباط بر بستر پروتکل اتصال گرا (UDP) شکل گرفته است. سوکت با استفاده از `socket.AF_INET` و `socket.SOCK_DGRAM` ساخته شده است. متغیر `UDP_IP` روی "0.0.0.0" تنظیم شده است؛ این مقدار دهی به سرور اجازه می‌دهد روی تمامی کارت های شبکه سیستم (چه وای فای و چه اترنت) به ترافیک ورودی روی پورت 5000 گوش دهد و وابستگی به یک IP ثابت محلی را از بین می‌برد.

۳. الگوریتم انباشت زیرپیکسل‌ها (Sub-pixel Accumulation): مهم‌ترین بخش منطق محاسباتی سرور، در مدیریت مقادیر حرکتی نهفته است. سیستم عامل‌ها برای جابه‌جایی مکان نما تنها مقادیر Integer را می‌پذیرند و خب این یکسری محدودیت حرکتی ایجاد میکند؛ بعنوان مثال، دیگر نمی‌توان ماوس را نیم پیکسل جابجا کرد. از طرفی، مقادیر تولید شده توسط ماتریس‌های چرخشی گوشه‌ای، اعداد اعشاری دقیق (Double) هستند. برای جلوگیری از هدر رفتن این حرکات ظریف (مثلاً حرکات‌های ۰.۳ پیکسلی) که به نوعی باعث روان تر شدن و تیز تر شدن حرکت ماوس می‌شود، دو متغیر سراسری `residual_x` و `residual_y` در نظر گرفته شده است. درون حلقه اصلی برنامه، ابتدا جابه‌جایی جدید با باقیمانده فریم قبلی جمع می‌شود (`raw_dx = command.get("DeltaX", 0) + residual_x`). سپس بخش صحیح آن استخراج شده (`move_x = int(raw_dx)`) و برای حرکت به سیستم‌عامل ارسال می‌گردد. در نهایت، بخش اعشاری مجدداً استخراج شده و در متغیر رزیدوال برای فریم بعدی ذخیره می‌شود (`residual_x = raw_dx - move_x`). این تکنیک باعث می‌شود حرکات بسیار کند و ظریف دست، روی هم انباشته شده و به محض رسیدن به یک پیکسل کامل، اعمال شوند؛ در نتیجه ماوس هرگز دچار لرزش یا پرش نمی‌شود.

```
# Store the fractional part for the next packet to avoid losing small movements
residual_x = raw_dx - move_x
residual_y = raw_dy - move_y
```

۴. مدیریت رویدادها (Event Handling) و همگام‌سازی (ACK): داده‌های دریافت شده در هر فریم تا حداکثر ۱۰۲۴ بایت (`recvfrom(1024)`) خوانده شده و پس از decode شدن از فرمت UTF-8، توسط کتابخانه `json` به یک دیکشنری پایتون تبدیل می‌شوند. سپس سه بخش کنترلی ارزیابی می‌شود:

- **مدیریت کلیک:** اگر کلید `Click` برابر ۱ باشد، تابع `pyautogui.click()` فراخوانی می‌شود. نکته ظریف مهندسی در این بخش، صفر کردن دستی متغیرهای `residual` بلافاصله قبل از کلیک است. این کار باعث می‌شود اگر در حین فشردن دکمه کلیک روی گوشه، دست کاربر لرزش میلی متری داشت (که خب این اتفاق به وفور رخ می‌دهد، چرا که برای انجام کلیک باید دست را به سمت چپ متمایل کرد و در حین این کار، دست حرکاتی به جهات راست و چپ و بالا و پایین نیز خواهد داشت)، آن لرزش اعمال نشود و ماوس دقیقاً در همان نقطه ثابت بماند و کلیک کند.

```
# If a click event is received, ignore any movement in the same packet
if click == 1:
    residual_x = 0.0
    residual_y = 0.0
    pyautogui.click()
    sock.sendto("ACK\n".encode('utf-8'), addr)
    continue
```

- **مدیریت حرکت:** اگر مقادیر صحیح استخراج شده (`move_x` یا `move_y`) صفر نباشند، تابع `moveRel` برای جابه‌جایی نسبی نسبت به موقعیت فعلی مکان‌نما اجرا می‌شود.
- **مدیریت اسکرول:** مقدار دریافتی در ضریب `SCROLL_MULTIPLIER` که برابر با ۱۲۰ است، ضرب می‌شود. عدد ۱۲۰ مقدار استاندارد API های ویندوز و لینوکس برای یک پله چرخش غلتک ماوس است. برای رویدادهای حیاتی یعنی کلیک و اسکرول، بلافاصله پس از اجرا، عبارت `"ACK\n"` رمزنگاری شده و با استفاده از تابع `sock.sendto` به آدرس فرستنده (گوشی) بازگردانده می‌شود تا قفل حرکتی سمت کلاینت باز شود.

۵. **بلوک پایداری نرم‌افزار:** کل فرآیند دریافت و پردازش داخل یک بلوک تو در تو `while True` و `try-except` محصور شده است. در صورتی که به دلیل نویز شبکه، یک بسته UDP ناقص به سرور برسد که قابل تبدیل به فرمت JSON نباشد (`json.JSONDecodeError`) و یا هر خطای پیش‌بینی نشده دیگری رخ دهد، برنامه از طریق دستور `pass` از آن چشم‌پوشی کرده و فوراً به سراغ دریافت بسته بعدی می‌رود. این معماری تضمین می‌کند که سرور تحت هیچ شرایطی متوقف نشده و همواره در دسترس باقی بماند.

ابزار Profile and Trace

بخش سوالات

برای پاسخ به این سوالات از پنج تا کوئری استفاده کردیم که در ادامه کد هر کدام را می‌بینیم و کاری که انجام می‌دهد را توضیح می‌دهیم. تمام این کدها در فایل `queries.sql` موجود هستند و خروجی آن‌ها در فایل‌های `query1.json`, `query2.json`, `query3.json`, `query4.json` و `query5.json` ذخیره شده‌اند.

```

SELECT
    name AS Task_Name,
    COUNT(*) AS Execution_Count,
    AVG(dur) / 1000000.0 AS Average_Duration_ms,
    MAX(dur) / 1000000.0 AS Max_Duration_ms
FROM slice
WHERE name = 'AirMouse_Filtering'
GROUP BY name;

```

این کوئری محاسبه می کند که تابع فیلترینگ در طول بازه 10 ثانیه ای trace چندبار اجرا شده و میانگین اجراها و حداکثر زمان اجرا را محاسبه می کند.

```

WITH SensorDeltas AS (
    SELECT
        name,
        ts,
        (ts - LAG(ts) OVER (PARTITION BY name ORDER BY ts)) / 1000000.0 AS delta_ms
    FROM slice
    WHERE name IN ('AirMouse_Gyro', 'AirMouse_Accel', 'AirMouse_Magneto')
)
SELECT
    name,
    COUNT(*) AS Total_Samples,
    AVG(delta_ms) AS Average_Sampling_Period_ms,
    1000.0 / AVG(delta_ms) AS Average_Frequency_Hz,
    MIN(delta_ms) AS Min_Delta_ms,
    MAX(delta_ms) AS Max_Delta_ms
FROM SensorDeltas
WHERE delta_ms IS NOT NULL
GROUP BY name;

```

این کوئری برای هر سنسور محاسبه می کند که چند بار اجرا شده، میانگین، حداقل، حداکثر زمان اجرا و میانگین فرکانس اجرایی که همان معکوس میانگین اجرا است را محاسبه می کند.

```

SELECT
    name AS Sensor_Task,
    SUM(dur) / 1000000.0 AS Total_CPU_Time_ms,
    AVG(dur) / 1000000.0 AS Avg_Time_Per_Execution_ms
FROM slice
WHERE name LIKE '%Sensor%' OR name LIKE '%Gyro%' OR name LIKE '%Accel%' OR name LIKE '%Magneto%'
GROUP BY name
ORDER BY Total_CPU_Time_ms DESC;

```

این کوئری برای هر سنسور محاسبه می کند که چه مدت زمانی روی CPU اجرا می شود و با تقسیم بر تعداد اجراهای هر سنسور (در مورد ما 480) میانگین را محاسبه می کند.

```
SELECT
  thread.name AS Thread_Name,
  thread_state.state AS State,
  SUM(thread_state.dur) / 1000000.0 AS Total_Duration_ms
FROM thread_state
JOIN thread USING (utid)
JOIN process USING (upid)
WHERE process.name = 'com.example.airmouse'
GROUP BY Thread_Name, State
ORDER BY Thread Name, Total Duration ms DESC;
```

این کوئری برای تردهای مختلف محاسبه می کند که چه مدت زمانی در وضعیت های مختلف هستند که در ادامه حالت های مختلف و معنی آن ها را ذکر می کنیم.

Running (۱)

- **مفهوم:** ترد در این حالت به طور فعال روی یکی از هسته های پردازنده (CPU) در حال اجراست و دستورات و محاسبات کدها را پردازش می کند. برای مثال، ترد اصلی ما ([xample.airmouse](#)) حدود ۳۰۴۳ میلی ثانیه مستقیماً روی CPU در حال کار بوده است.

Runnable یا R (۲)

- **مفهوم:** ترد کاملاً آماده است، هیچ مانعی برای اجرا ندارد و منتظر دیتایی نیست، اما به دلیل اینکه تمام هسته های CPU توسط تردهای دیگر اشغال شده اند، در صف زمان بندی منتظر خالی شدن CPU مانده است. در کد ما بالا بودن این زمان (مانند معطلی ۶۸۷ میلی ثانیه ای ترد اصلی یا ۵۱۴ میلی ثانیه ای ترد JIT) نشان دهنده رقابت شدید تردها بر سر تصاحب هسته های پردازنده است.

Preempted یا +R (۳)

- **مفهوم:** این حالت زیرمجموعه‌ای از وضعیت Runnable است. ترد در حال اجرای کدهای خود روی CPU بوده، اما زمان‌بند سیستم‌عامل به طور ناگهانی اجرای آن را قطع کرده و به صف انتظار فرستاده است. این اتفاق معمولاً به دو دلیل رخ می‌دهد:
 1. سهمیه زمانی (Time Slice) ترد برای استفاده از CPU تمام شده است.
 2. یک ترد دیگر با اولویت بالاتر بیدار شده و CPU را از این ترد گرفته است.

۴ S یا Interruptible Sleep

- **مفهوم:** ترد کار فعلی خود را تمام کرده و به طور داوطلبانه به حالت خواب یا انتظار رفته است تا یک رویداد (Event) خاص رخ دهد. این رویداد می‌تواند فعال شدن یک تایمر، بیدار شدن توسط قفل‌های برنامه (Mutex/Lock)، فرستادن پکت جدید، یا منتظر ماندن برای دریافت داده از سنسور یا سیگنال گرافیکی (Vsync) باشد. این حالت با سیگنال‌های سیستم‌عامل به راحتی بیدار و به حالت Runnable تبدیل می‌شود. طبیعی است که تردهایی مثل **RenderThread** یا **GPU completion** بخش عمده زمان خود را (بیش از ۸۰۰۰ میلی‌ثانیه) در این حالت بگذرانند، چون فقط زمان‌هایی که دیتای جدید سنسور می‌رسد یا فریم جدیدی باید رسم شود بیدار می‌شوند و بقیه زمان را منتظر می‌مانند.

۵ D یا Uninterruptible Sleep

- **مفهوم:** ترد در سطح هسته سیستم‌عامل (Kernel Level) به دلیل انتظار برای یک منبع سخت‌افزاری مستقیم یا عملیات ورودی/خروجی (I/O) مسدود (Block) شده است. تفاوت آن با حالت **S** این است که این ترد به قدری درگیر پاسخ سخت‌افزار است که حتی سیستم‌عامل با سیگنال‌های عادی هم نمی‌تواند آن را وسط کار بیدار یا مجبور به توقف کند و باید حتماً کار درایور تمام شود. این وضعیت معمولاً به دلیل فراخوانی‌های سیستمی (System Calls) مربوط به دیسک یا شبکه رخ می‌دهد. در خروجی ما، ترد شبکه (**-pool-10-thread**) و درایور گرافیک (**mali-cmar-backe**) دچار این خواب سخت‌افزاری شده‌اند که نشان‌دهنده معطلی آن‌ها در لایه درایور سخت‌افزار است.

```
SELECT
  name,
  ts / 1000000000.0 AS Start_Time_Seconds,
  dur / 1000000.0 AS Duration_ms
FROM slice
WHERE name = 'AirMouse_SocketSend'
ORDER BY dur DESC
LIMIT 20;
```

این کوئری تمام نمونه های اجرای تابع ارسال بسته UDP را بر اساس زمان اجرا از بیشتر به کمتر مرتب می کند و به ما برمیگرداند.

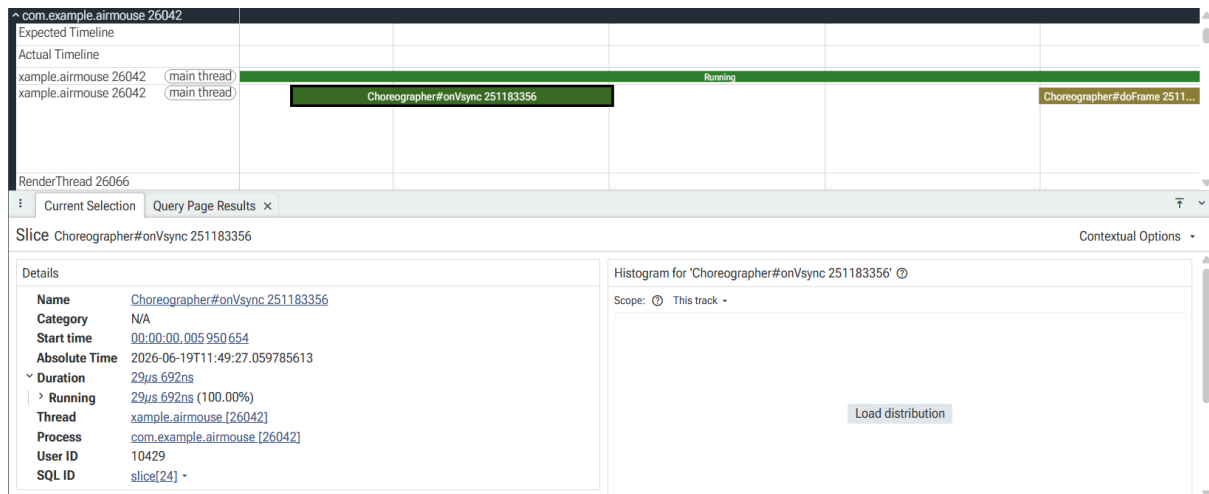
۱) روند سیستم عامل از درخواست تا دریافت داده های سنسور

با تفکیک دقیق فرآیند اختصاصی اپلیکیشن `com.example.airmouse` و حذف نویزهای سیستم عامل، لایف ساینکل و خط لوله (Pipeline) تولید تا رندر یک فریم مشخص به طور کامل شفاف شده است. این فرآیند به صورت متوالی و موازی بین دو تِرد اصلی و رندر جریان دارد که مستندات زمانی و ساختاری آن بر اساس داده های تریس به شرح زیر است:

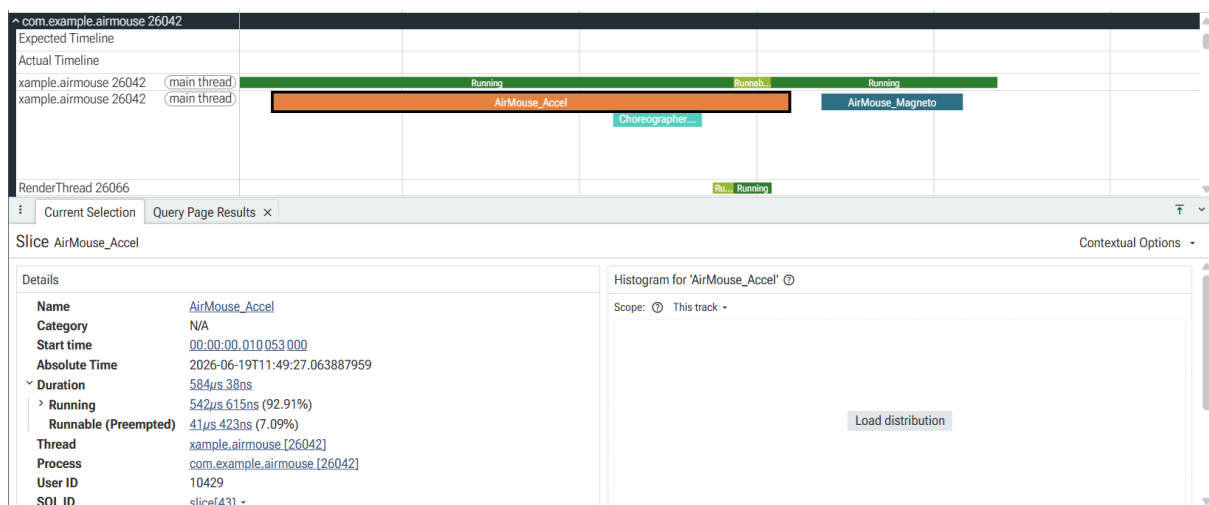
بخش اول: فازهای پردازش در تِرد اصلی (`UI Thread - TID: 26042` یا همان `xample.airmouse`)

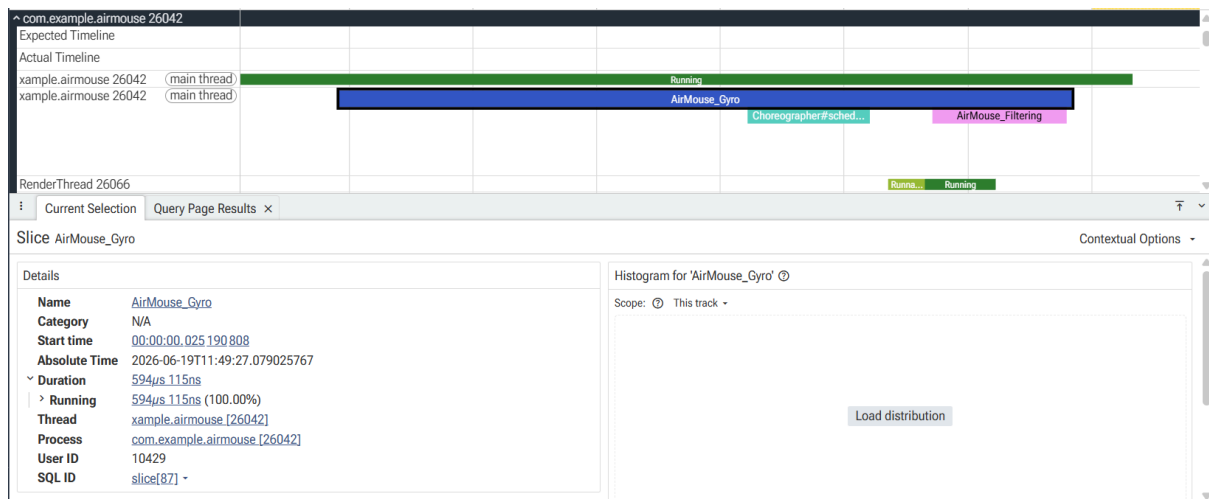
تِرد اصلی وظیفه دریافت سیگنال، پردازش داده های حسگر و آماده سازی ساختار درخت ویوها (`DisplayList`) را بر عهده دارد:

- **دریافت و هوشیاری سیگنال همگام سازی (`Vsync`):** سیستم اندروید با ارسال سیگنال `Vsync` به اپلیکیشن اعلام می کند که پنجره زمانی برای آماده سازی فریم جدید آغاز شده است. تِرد اصلی در وضعیت `Running` قرار گرفته و پردازش را آغاز می کند.

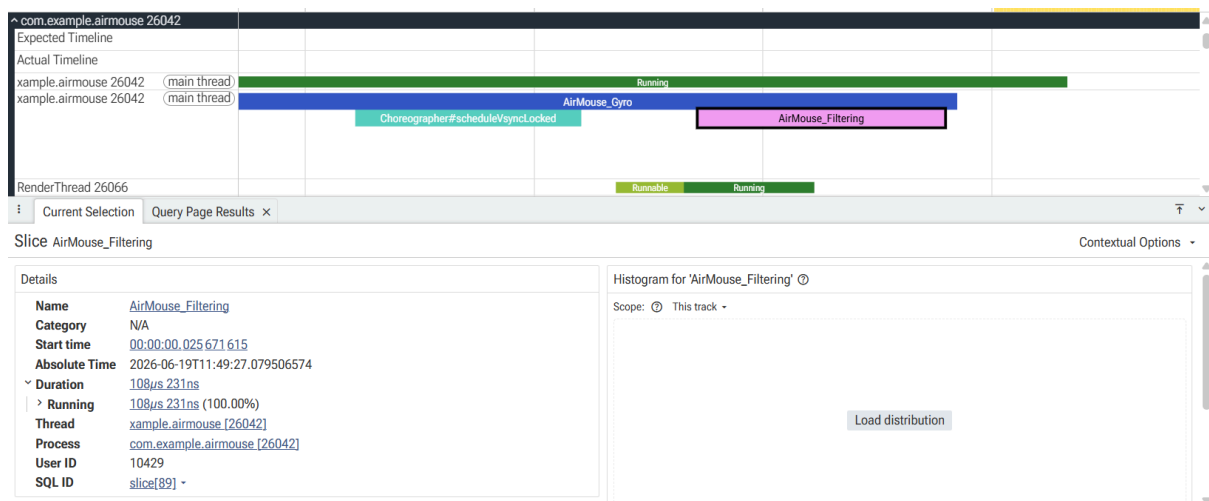


- پردازش داده‌های ورودی حسگر (Sensor Processing): برنامه دیتای خام شتاب‌سنج،ژیروسکوپ و مغناطیس‌سنج را به ترتیب از سنسور فرام‌ورک دریافت می‌کند. میانگین زمان پردازش خام برای **AirMouse_Gyro** حدود **1.07 میلی‌ثانیه**، برای **AirMouse_Accel** حدود **0.69 میلی‌ثانیه** و برای **AirMouse_Magneto** حدود **0.39 میلی‌ثانیه** است.

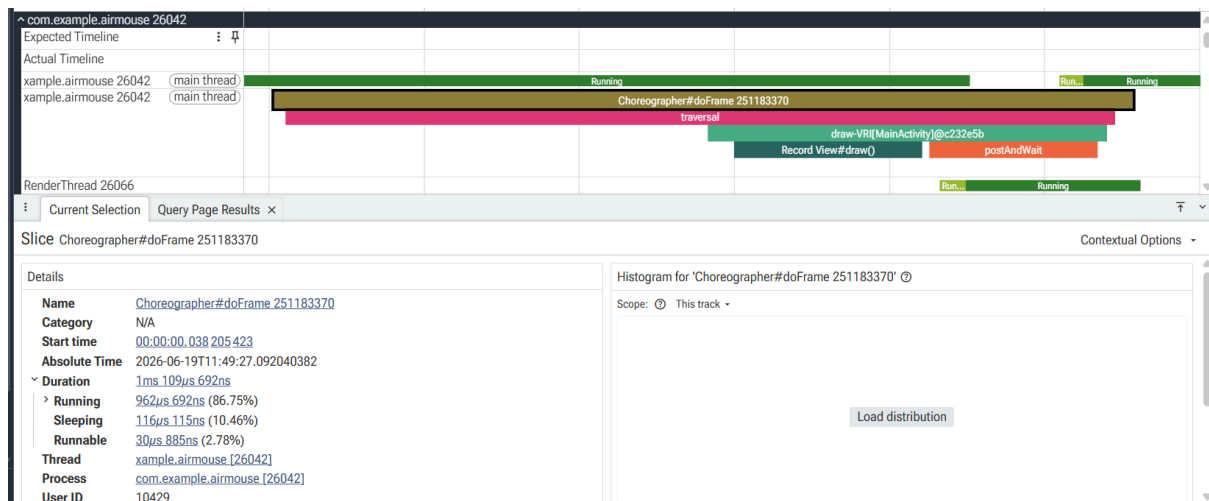




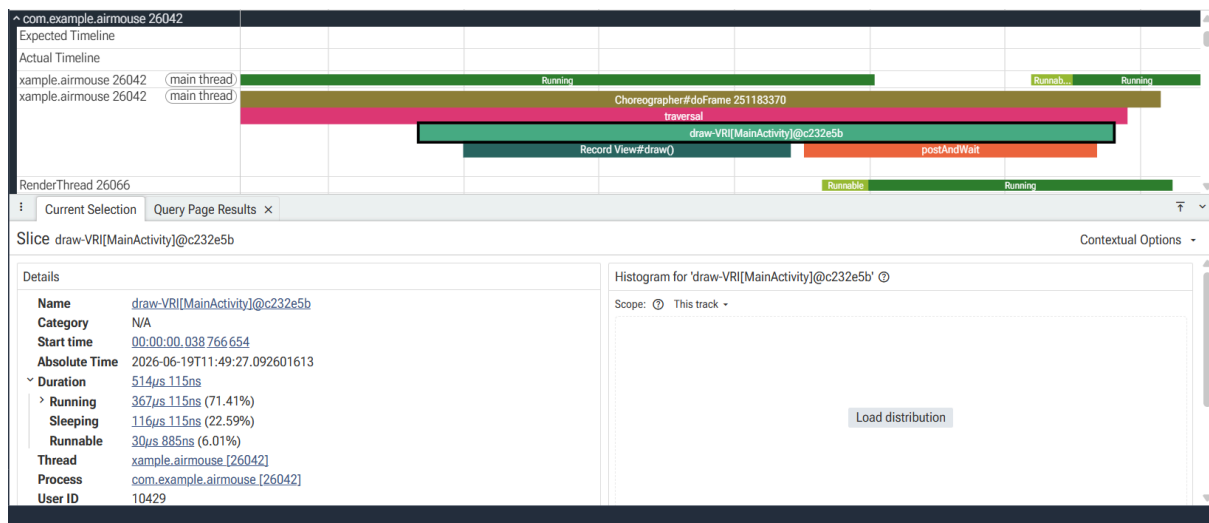
- فاز فیلترینگ و بهروزرسانی وضعیت: بلافاصله اسلایس **AirMouse_Filtering** با میانگین زمان 0.26 میلی ثانیه اجرا می شود تا نویز داده ها حذف و موقعیت نشانگر به روز شود.



- آغاز چرخه **doFrame** و **Traversal**: بدنه اصلی گرافیکی درون متد **Choreographer#doFrame** آغاز شده و فاز طولانی **traversal** شامل محاسبات ابعاد (Measure) و موقعیت دهی هندسی ویوها (Layout) روی صفحه انجام می شود.

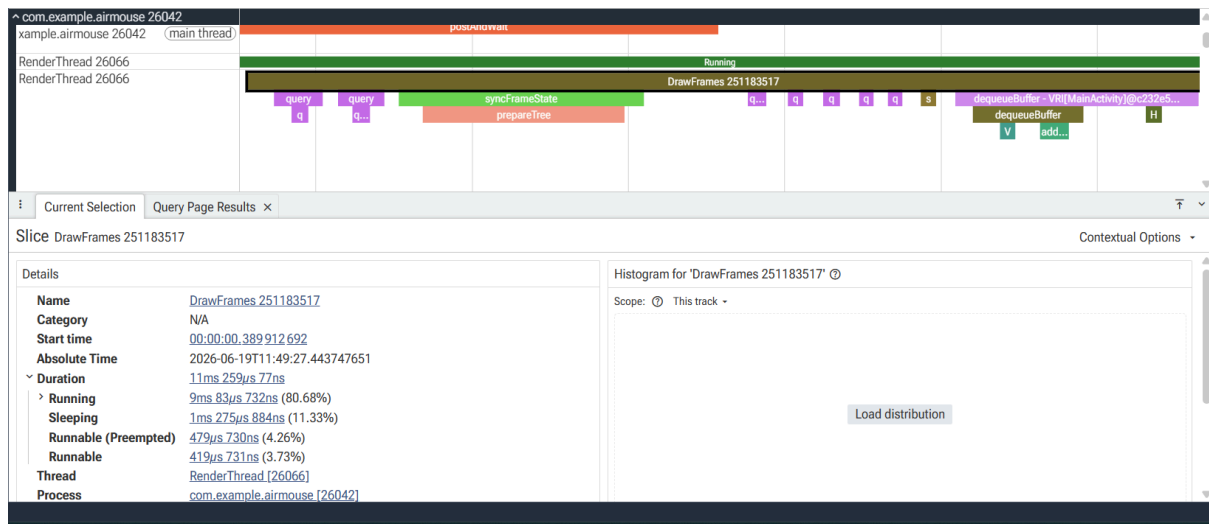


- فاز ضبط دستورات رسم (Draw & Record): متد داخلی `Record View#draw()` فراخوانی می‌شود تا تمام کارهای گرافیکی در قالب یک لیست از دستورات (`DisplayList`) بدون رندر واقعی روی کارت گرافیک، ضبط و ذخیره شوند. پس از پایان این فاز، وظیفه تِرد اصلی به اتمام می‌رسد.

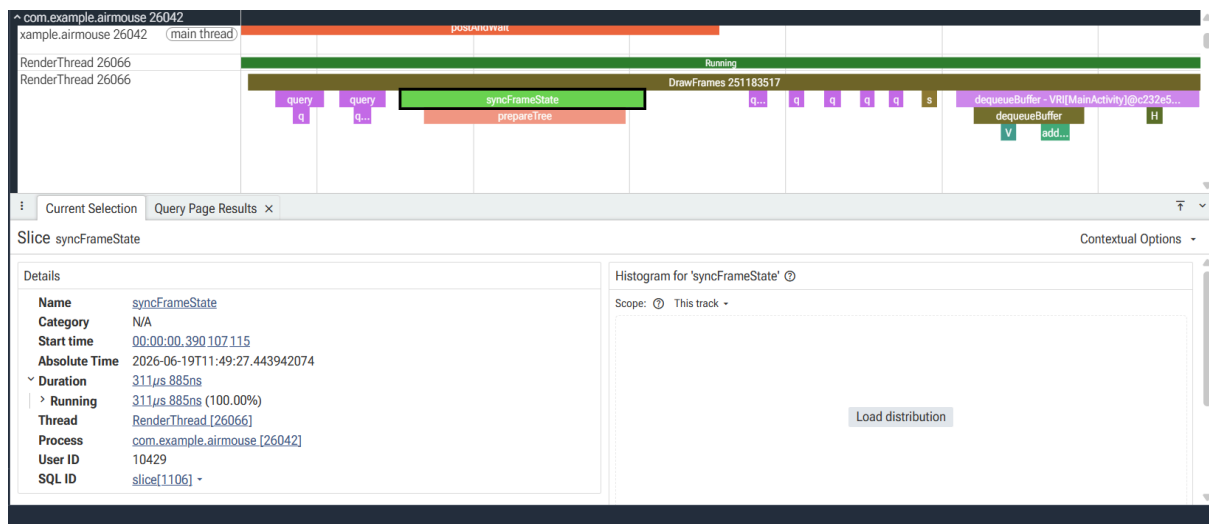


بخش دوم: فازهای پردازش در تِرد رندر (`RenderThread - TID: 26066`) به محض آماده شدن `DisplayList` در تِرد اصلی، `RenderThread` فعال شده و عملیات رندر سخت‌افزاری را پیش می‌برد:

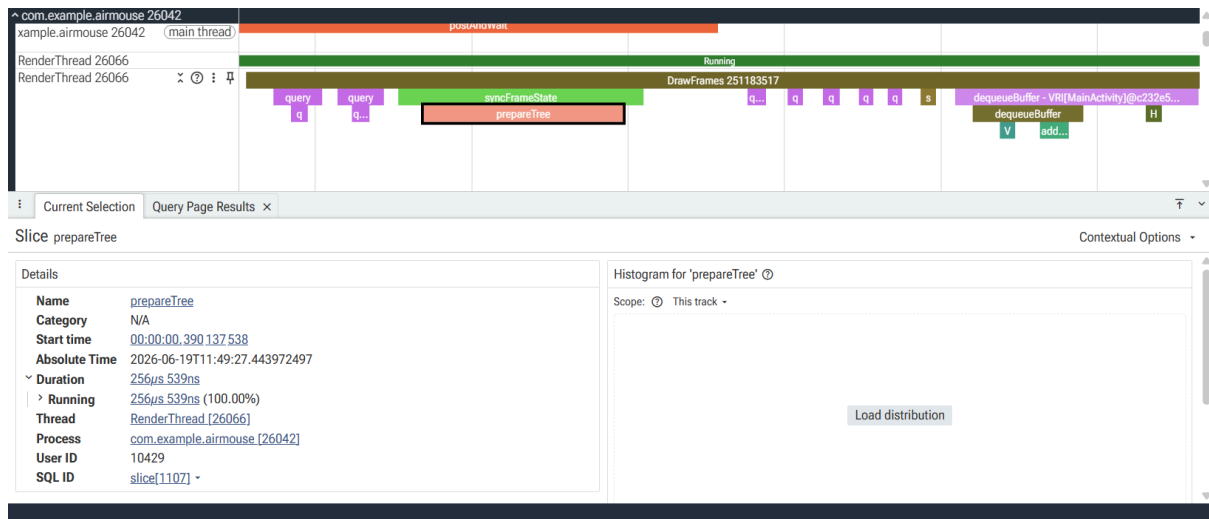
- تحویل گرفتن فریم (`DrawFrames`): تِرد رندر بیدار شده و فرآیند پردازش کارت گرافیک را آغاز می‌کند.



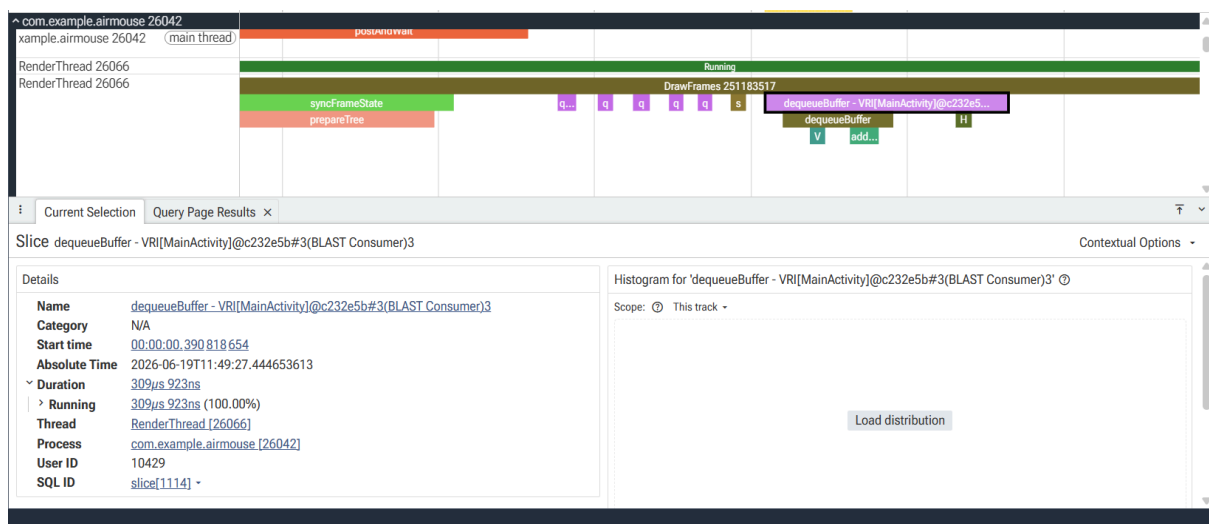
- همگام‌سازی اطلاعات فریم (**syncFrameState**): داده‌های ضبط شده در فاز Draw لایه کاربر، به صورت ایمن به حافظه رندر سخت‌افزاری کپی می‌شوند.



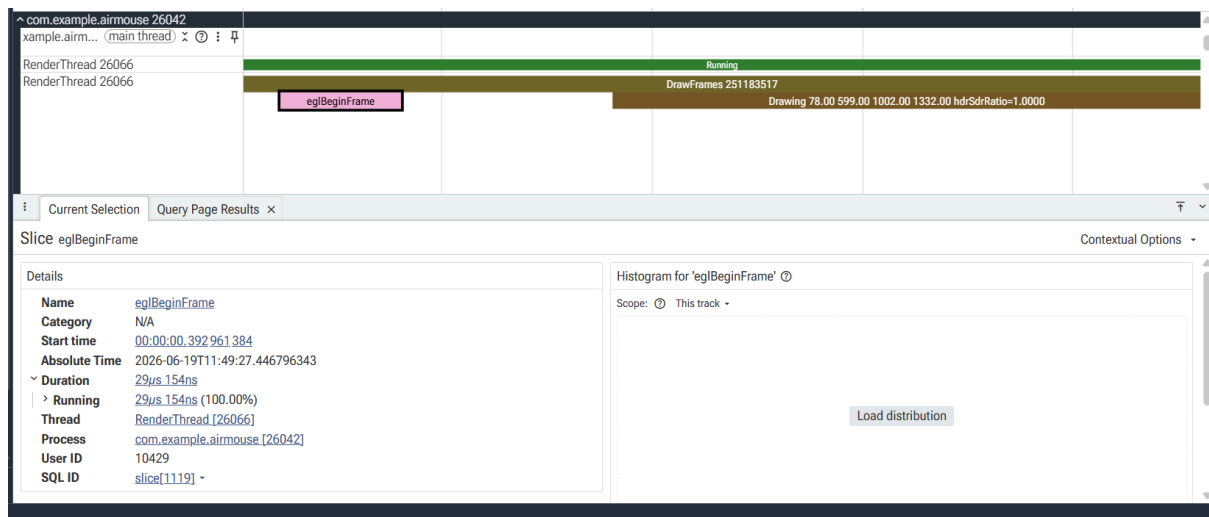
- آماده‌سازی درخت لایه‌های گرافیکی (**prepareTree**): کل زنجیره درخت ویوها ارزیابی شده و لایه‌ها آماده رندر می‌شوند.



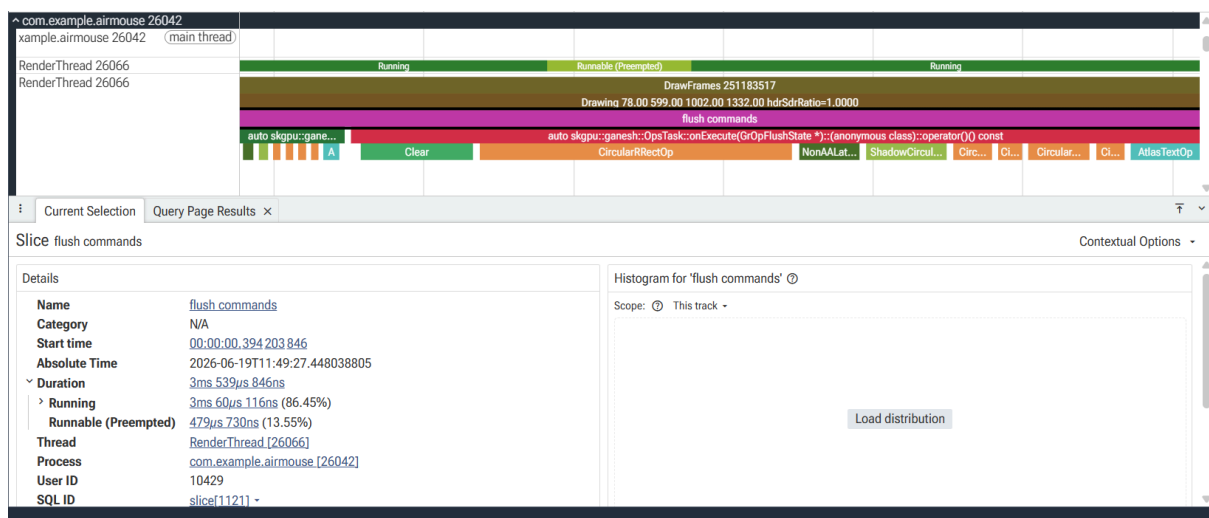
- درخواست بافر از سیستم عامل (**dequeueBuffer**): ترد رندر یک بافر خالی گرافیکی از **SurfaceFlinger** سیستم عامل درخواست می کند تا تصویر جدید را درون آن نقاشی کند.



- صدور دستورات رسم روی **GPU**: دستورات نهایی رندرینگ از طریق لایه درایور (**eglBeginFrame** و اسلایس **Drawing**) به زبان پردازنده گرافیکی ترجمه شده و پیکسل ها روی بافر ترسیم می شوند.



- **ثبت نهایی و فلاش کردن دستورات:** بافر پر شده از طریق دستورات فلاش به صف نمایشگر فیزیکی (Display) فرستاده می‌شود.



اثبات مکانیزم خط لوله (Pipeline): این دنباله به وضوح نشان می‌دهد که به محض پایان فاز ضبط در تِرد اصلی، این تِرد کاملاً آزاد شده و مجموعاً **6150.16 میلی‌ثانیه** را در وضعیت خواب (S) می‌گذراند تا سیگنال‌های فریم‌های بعدی را دریافت کند، در حالی که **RenderThread** به صورت موازی و مستقل در حال پیشبرد عملیات گرافیکی روی سخت‌افزار است.

۲) کارکرد سنسورها، خطای سنسور خام و فرآیند دقیق ترکیب آن‌ها (Sensor Fusion)

- **شتاب‌سنج (Accelerometer):** شتاب خطی مکانیکی وارد بر دستگاه را در سه محور خروجی می‌دهد. این سنسور به شدت تحت تأثیر نیروی گرانش زمین قرار دارد و به دلیل

لرزش‌های فیزیکی ناخواسته دست کاربر، نویزهای فرکانس بالا (High-Frequency Noise) تولید می‌کند که باعث پرش مکرر cursor در فواصل کوتاه می‌شود.

- **ژیروسکوپ (Gyroscope):** سرعت زاویه‌ای (دوران) را پیرامون سه محور اندازه‌گیری می‌کند. برای تبدیل سرعت زاویه‌ای به زاویه مطلق جهت حرکت موس، باید روی داده‌های آن انتگرال‌گیری زمانی انجام شود. این فرآیند ریاضی خطای بایاس ناچیز سخت‌افزاری را در طول زمان انباشته کرده و پدیده **دریفت (Drift)** یا خزش مداوم و خودبه‌خودی مکان‌نما را رقم می‌زند.

- **مغناطیس‌سنج (Magnetometer):** راستای میدان مغناطیسی زمین را برای کالیبره کردن جهت جغرافیایی (Yaw) می‌سنجد، اما به شدت به نویزهای فرکانس پایین محیطی (فلزات اطراف، امواج الکترومغناطیسی گوشی) حساس است.

- **معماری زنجیره فیلترینگ استفاده شده:** برای مهار این خطاهای ذاتی، خط لوله پردازش داده ما از یک مدل سه‌مرحله‌ای استفاده کرده است:

1. **فیلتر مکمل (Complementary Filter):** با فرمولاسیون تخصیص وزن ۹۸ درصد به

داده انتگرال‌گیری‌شده ژيروسکوپ و ۲ درصد به زاویه سنجیده شده توسط شتاب‌سنج عمل می‌کند. این فیلتر به صورت همزمان نقش یک فیلتر بالاگذر را برای ژيروسکوپ (حفظ پاسخ سریع به حرکت‌های آنی و حذف دریفت فرکانس پایین) و فیلتر پایین‌گذر را برای شتاب‌سنج (استفاده از پایداری گرانش در بلندمدت و حذف نویز ارتعاشی فرکانس بالا) ایفا کرده و یک زاویه پایدار تولید می‌کند. ریاضیات فیلتر مکمل به طور همزمان به عنوان یک **فیلتر بالاگذر (High-Pass)** برای ژيروسکوپ و یک **فیلتر پایین‌گذر (Low-Pass)** برای شتاب‌سنج عمل می‌کند:

- وزن ۹۸٪ به ژيروسکوپ یعنی سیستم در کوتاه‌مدت (تکان‌های سریع و آنی دست) کاملاً به ژيروسکوپ اعتماد می‌کند؛ چون ژيروسکوپ تحت تأثیر لرزش‌های خطی قرار نمی‌گیرد و پاسخ بسیار سریع و بدون تأخیری دارد.

- وزن ۲٪ به شتاب‌سنج یعنی سیستم در بلندمدت به آرامی و به صورت مداوم، خروجی را با بردار گرانش شتاب‌سنج همگام می‌کند. این ۲٪ مانند یک لنگرگاه عمل می‌کند که اجازه نمی‌دهد خطای انتگرال‌گیری ژيروسکوپ (دریفت) به سمت بی‌نهایت رشد کند و مدام آن را به سمت زاویه واقعی زمین کالیبره می‌کند.

2. **فیلتر پایین‌گذر ثانویه (Low-Pass Filter):** پس از فیلتر مکمل، ممکن است

همچنان پله‌های فرکانسی بسیار کوچک ناشی از تبدیل دیجیتال به آنالوگ در سخت‌افزار (Quantization Noise) یا نوسانات فرکانس بالای باقیمانده وجود داشته باشند. فیلتر پایین‌گذر ثانویه با حذف این فرکانس‌های تیز مزاحم، لبه‌های

حرکتی سیگنال را کاملاً نرم (Smooth) می‌کند. در نتیجه، حرکت مکان‌نما روی صفحه کاملاً روان، خطی و پیوسته دیده می‌شود.

3. **فیلتر آستانه حرکت (Deadzone/Threshold Filter):** دست انسان حتی زمانی که تلاش می‌کند کاملاً ثابت بماند، دارای یک لرزش بیولوژیکی ناخودآگاه (Physiological Tremor) با فرکانسی بین ۸ تا ۱۲ هرتز است. سنسورهای میکروالکترومکانیکی (MEMS) گوشی این لرزش‌های میکروسکوپی را هم ثبت می‌کنند. اگر فیلتر آستانه نباشد، وقتی موس روی میز یا دست ثابت است، مکان‌نما روی مانیتور به طور مداوم تیک می‌زند یا می‌لرزد. فیلتر آستانه حرکت (Deadzone) با تعریف یک مرز دیجیتالی (مثلاً اگر تغییر زاویه خروجی کمتر از یک حد مشخص بود، حرکت را صفر فرض کن)، این نویز لرزش دست در حالت سکون را کاملاً قیچی می‌کند تا مکان‌نما در زمان عدم حرکت کاربر، پایداری مطلق داشته باشد.

۳) مقایسه زمان بین دو داده متوالی (Perfetto) با دوره نمونه‌برداری نامی

تطابق میان کد و واقعیت سیستم‌عامل به شکل زیر تحلیل می‌شود:

- **در سطح کد:** با توجه به فرکانس هدف ~۵۰ هرتز، دوره نمونه‌برداری ایده‌آل برنامه‌ریزی شده برابر با ۲۰.۰۰ میلی‌ثانیه است.
- **در سطح سیستم‌عامل واقعی (کوئری ۲):**
 - دوره تناوب میانگین برای هر سه سنسور تقریباً 20.61 (ms) (معادل فرکانس میانگین 48.51 (Hz)) ثبت شده است که بسیار نزدیک به مقدار پیکربندی شده در کد است.
 - **تحلیل نوسان (Jitter):** اما فواصل لحظه‌ای بین داده‌ها اصلاً ثابت نیست.
 - در شتاب‌سنج، فاصله متوالی بین حداقل 13.12 (ms) و حداکثر 29.04 (ms) نوسان دارد.
 - در ژيروسکوپ این بازه بین 13.52 (ms) تا 27.17 (ms) است.
 - در مغناطیس‌سنج این بازه بین 12.71 (ms) تا 29.23 (ms) است.

علت تفاوت: سیستم‌عامل اندروید یک سیستم‌عامل بی‌درنگ (Real-Time OS) نیست. فرآیندهایی مثل زمان‌بندی هسته لینوکس، نحوه صف‌بندی داده‌ها در لایه انتزاع سخت افزار (Sensor Hardware Abstraction Layer) و تاخیر در مکانیزم ارتباط بین‌فرآیندی (Binder IPC)

باعث می‌شود که نمونه‌ها با فواصل نامنظم و همراه با Jitter بالا به دست اپلیکیشن برسند، هرچند که میانگین کل آن‌ها همان حدود ۲۰ میلی‌ثانیه حفظ می‌شود.

۴) آیا هنگام استفاده از فراخوانی‌های سیستمی (System Calls)، میان فعالیت‌های حساس به زمان (مانند به‌روزرسانی سنسورها) و پردازش‌های سنگین (مانند رندر گرافیکی)، تعارضی رخ می‌دهد؟ پاسخ خود را توجیه کنید.

بله، تعارض و رقابت شدید رخ می‌دهد. برای اثبات این موضوع، داده‌های مربوط به وضعیت (State) تردها در لایه سیستم‌عامل را بررسی می‌کنیم:

۱. رقابت بر سر هسته‌های پردازنده (CPU Contention): ترد اصلی اپلیکیشن (xample.airmouse) که کارهای حساسی مثل دریافت رویداد سنسورها و فیلترینگ را انجام می‌دهد، مدت زمان طولانی (687.42 ms) را در وضعیت *Runnable (R)* سپری کرده است. این یعنی ترد آماده اجرا بوده اما مجبور شده در صف زمان‌بندی هسته معطل بماند؛ چرا که پردازش‌های سنگین دیگری مانند *RenderThread* (با زمان اجرای فعال 1160.64 ms) و ترد پردازشی ماشین مجازی *Jit thread pool* (با زمان اجرای فعال 539.47 ms) در حال مصرف منابع CPU بوده‌اند.

۲. بلاک شدن در لایه درایور و فراخوانی سیستمی: ترد ارتباطی شبکه (*-pool-10-thread*) و درایور گرافیک (*mali-cmar-backe*) به ترتیب (0.883 ms) و (0.901 ms) را در وضعیت خطرناک *Uninterruptible Sleep (D)* گذرانده‌اند. این وضعیت مستقیماً زمانی رخ می‌دهد که یک ترد یک فراخوانی سیستمی مسدودکننده (مانند نوشتن روی سوکت شبکه یا دستورات ioctl سخت‌افزار) انجام می‌دهد و تا زمان آزاد شدن منابع سخت‌افزاری در سطح هسته سیستم‌عامل، متوقف می‌شود. این قفل شدن‌ها می‌توانند به طور غیرمستقیم از طریق قفل‌های اشتراکی (Mutexes)، تردهای دیگر (حتی ترد اصلی) را منتظر نگه‌دارند.

۵) تفاوت سنسورها به صورت Wake-up و Non-wake-up

این تفاوت مربوط به مدیریت توان (Power Management) در معماری مجتمع SoC دستگاه است:

- **سنسورهای Wake-up:** این سنسورها به خط وقفه اختصاصی (Wakeup Interrupt Line) متصل هستند که می‌تواند پردازنده اصلی (Application Processor - AP) گوشی را از وضعیت خواب عمیق یا تعلیق (Suspend State) بیدار کند.
 - مزایا: تضمین عدم گم شدن رویدادها در زمان قفل بودن گوشی. برای کاربردهای حیاتی مانند پایش تصادفات، سقوط آزاد یا گام‌شمار پس‌زمینه الزامی است.

- معایب: مصرف باتری را به شدت افزایش می‌دهد، چرا که بیدار کردن مکرر پردازنده از خواب عمیق مانع از رفتن سیستم به حالت بهینه توان می‌شود.
- **سنسورهای Non-wake-up:** رویدادهای این سنسورها توانایی بیدار کردن AP را ندارد. داده‌های تولیدی یا در بافر سخت‌افزاری (FIFO سنسور) ذخیره شده و پس از پر شدن رونویسی می‌شوند یا کلاً می‌سوزند.
- مزایا: صرفه‌جویی فوق‌العاده در مصرف انرژی فیزیکی باتری؛ سیستم‌عامل با خیال راحت در خواب می‌ماند.
- معایب: در صورت خاموش شدن صفحه نمایش یا خواب رفتن پردازنده، اپلیکیشن دسترسی زنده به داده‌ها را از دست می‌دهد.
- نتیجه برای پروژه: برای اپلیکیشن موس هوایی که صرفاً در زمان روشن بودن صفحه و تعامل فعال کاربر کاربرد دارد، استفاده از نوع **Non-wake-up** صد درصد منطقی و بهینه است.

۶) مدت زمانی را که تابع فیلتر در سطح CPU اجرا می‌شود را به صورت میانگین به دست آورید.

طبق داده‌های مستقیم کوئری اول مربوط به برش زمان اجرای الگوریتم فیلترینگ:

- **میانگین زمان اجرا:** $0.2687 (ms)$ (حدود ۲۶۸ میکروثانیه).
- نکته تحلیلی: بدترین زمان (طولانی ترین) اجرای این تابع $1.295 (ms)$ ثبت شده که نشان می‌دهد حتی یک تابع فیلتر محاسباتی بهینه نیز به دلیل پدیده‌های سیستمی (مانند اختلال در حافظه کش یا جابجایی ترد بین هسته‌ها) می‌تواند گاهی تا ۵ برابر زمان عادی خود طول بکشد.

۷) بیشترین مصرف پردازشی مربوط به کدام سنسور بوده است؟

با مقایسه زمان خالص مصرف پردازنده در کوئری مربوط به سنسورها:

1. سنسور ژيروسکوپ (**AirMouse_Gyro**): مجموعاً $517.874 (ms)$ (میانگین هر بار اجرا: $1.076 (ms)$)
2. سنسور شتاب‌سنج (**AirMouse_Accel**): مجموعاً $333.686 (ms)$ (میانگین هر بار اجرا: $0.693 (ms)$)
3. سنسور مغناطیس‌سنج (**AirMouse_Magneto**): مجموعاً $188.888 (ms)$ (میانگین هر بار اجرا: $0.392 (ms)$)

نتیجه: بیشترین مصرف پردازشی مستقیماً متعلق به سنسور ژيروسکوپ است که هم زمان پردازش کل بالاتری داشته و هم میانگین زمان پردازش هر نمونه‌ی آن بیش از دو برابر مغناطیس‌سنج است. دلیل آن فرآیندهای محاسباتی سنگین‌تر لایه‌های پائین برای تبدیل سرعت زاویه‌ای کوئری‌ها است.

۸) نرخ نمونه‌برداری سنسورها چه تأثیری بر سایر پردازش‌های سیستمی دارد؟

افزایش نرخ نمونه‌برداری (مثلاً تغییر از ۵۰ هرتز به ۱۰۰ یا ۲۰۰ هرتز) تأثیرات مخرب زیر را بر کل سیستم می‌گذارد:

۱. **افزایش بار ترد اصلی (UI Thread Overhead):** با بالا رفتن نرخ، تعداد فراخوانی‌های تابع `onSensorChanged` روی ترد اصلی به شدت افزایش می‌یابد. از آنجا که پردازش سنسورها در حال حاضر نیز بخش بزرگی از زمان اجرای فعال ترد اصلی (3043.65 ms) را به خود اختصاص داده، نرخ بالاتر باعث قفل شدن بیشتر این ترد می‌شود.

۲. **تشدید رقابت صف پردازنده (CPU Contention):** افزایش فراخوانی‌ها باعث می‌شود تردهایی مثل `xample.airmouse` زمان بیشتری را در وضعیت `Runnable (R)` (که الان ۶۸۷ میلی‌ثانیه است) منتظر بمانند. این امر زمان پردازنده را از تردهای حساسی مثل `RenderThread` می‌رباید و مستقیماً منجر به افت فریم شدید در محیط کاربری اندروید می‌شود.

۳. **اشباع لایه ارتباطی و شبکه:** نرخ نمونه‌برداری بالا یعنی ارسال بسته‌های UDP بیشتر در ثانیه. این موضوع ترد شبکه (`-pool-10-thread`) را شلوغ‌تر کرده و به دلیل پر شدن بافرهای سوکت سیستم‌عامل، زمان‌های بلاک لایه هسته (وضعیت `D`) و تاخیرهای ارسالی شبکه را به شدت بالا می‌برد.

۹) از آماده شدن اطلاعات سنسور تا ارسال به سرور/رایانه و مشاهده حرکت مکان نما متوسط چقدر زمان مصرف می‌شود؟

برای محاسبه این تاخیر انتهای-به-انتهای (End-to-End Latency)، زنجیره زمانی زیر را بر اساس آمار داده‌ها جمع می‌زنیم:

- پردازش سنسور در برنامه: میانگین زمان خواندن و آماده‌سازی دیتای سنسورها روی پردازنده حدود 0.7 ms است.
- الگوریتم فیلترینگ: اجرای تابع `AirMouse_Filtering` به طور میانگین 0.268 ms طول می‌کشد.

3. تاخیر ارسال سیستم عامل (UDP Send Execution): طبق کوئری آخر، در حالت عادی و بدون در نظر گرفتن پیک‌های سنگین، اکثر ارسال‌های تابع `AirMouse_UDP_Send` زمان اجرایی بین 3.5 (ms) تا 4.5 (ms) دارند (میانگین حدود 4 (ms)).
4. تاخیر انتقال شبکه (Wi-Fi Propagation): در یک شبکه محلی (LAN) بی‌سیم پایدار، زمان پینگ و انتقال بسته UDP تا کامپیوتر معمولاً بین 2 (ms) تا 5 (ms) است.
5. پردازش و رندر سمت سرور (PC Side): دریافت پکت توسط اسکریپت پایتون، پردازش آن و اعمال حرکت مکان‌نما از طریق کتابخانه‌هایی مثل PyAutoGUI و رندر سیستم عامل مقصد، حدود 5 (ms) تا 15 (ms) زمان می‌برد.

مجموع تاخیر متوسط: با جمع این مقادیر، زمان متوسط کلی از لحظه آماده شدن سنسور تا حرکت مکان‌نما در شرایط عادی حدود 15 (ms) تا 25 (ms) ارزیابی می‌شود.

- نکته مهم: اگر سیستم عامل دچار لگ درایور شبکه شود (پیک‌های $26..76\text{ (ms)}$ که در لیست تاخیرهای ارسال UDP ما دیده می‌شود)، این تاخیر کل می‌تواند لحظه‌ای به بیش از 50 (ms) برسد که برای کاربر به صورت پریدن ناگهانی موس حس خواهد شد.

۱۰ thread های اصلی، thread های سنسور، thread های ارتباطی، پردازشی و UI چگونه تفکیک می‌شوند؟ بررسی شود که هر سنسور چه نوع event هایی تولید می‌کند، این event ها روی کدام thread ها اجرا می‌شوند و چه تأثیری بر Main Thread دارند؟

تفکیک تردها در فرآیند برنامه ما براساس داده‌های Perfetto:

- **Main Thread**: با نام `xample.airmouse` در فرآیند مشخص است. مسئول مدیریت لایف ساینل برنامه، دریافت رویدادهای ورودی و هماهنگی ترسیم‌ها است.
- **Sensor/IPC Threads**: تردهای سیستم عاملی بایندر تحت عنوان `binder:26042_5/6/7` که وظیفه جابجایی داده‌های سنسور از سرویس سیستم عامل به داخل فرآیند برنامه را دارند.
- **Render Thread**: تحت نام `RenderThread` کار صدور دستورات نهایی گرافیکی به پردازنده گرافیکی (Mali) را انجام می‌دهد.
- **Communication Thread**: ترد ورکر شبکه با نام `pool-10-thread` که ارسال پکت‌های شبکه را به صورت موازی جلو می‌برد.

نحوه تولید و جریان رویدادها (Event Flow):

1. سنسورهای شتاب‌سنج، ژيروسکوپ و مغناطیس‌سنج در سطح سخت‌افزار رویدادهای تغییر مقادیر فیزیکی ([SensorEvent](#)) را تولید می‌کنند.
2. این رویدادها توسط تردهای بایندر سیستم ([binder](#)) دریافت و به صف پیام‌های ترد اصلی برنامه ارسال می‌شوند.
3. تابع کال بک ([onSensorChanged](#)) برای هر سنسور روی ترد اصلی (Main Thread) اجرا می‌شود.

تأثیر بر Main Thread: از آنجا که این ایونت‌ها با فرآیند فیلترینگ به طور مستقیم روی ترد اصلی فراخوانی می‌شوند، بار پردازشی عظیمی به آن تحمیل می‌کنند (بیش از ۱۰۰۰ میلی‌ثانیه زمان خالص اجرای توابع سنسورها). این موضوع باعث می‌شود ترد اصلی توانایی کمتری برای رسیدگی به کارهای گرافیکی و تعاملی داشته باشد و به همین دلیل زمان معطلی آن در صف پردازنده افزایش یافته است (687.42 (ms) در وضعیت R)؛ اما به دلیل مجزا بودن ترد شبکه ([pool-10-thread](#) -)، کارهای طولانی ارسال UDP به ترد اصلی سرایت نکرده و آن را کاملاً مسدود نمی‌کند.

۱۱) تفاوت حرکت آرام و ناگهانی تلفن همراه روی پردازش‌های صورت گرفته چگونه است؟ آیا تأخیر حرکت مکان نما ماوس در دو حالت تفاوتی دارد؟

- از نظر میزان محاسبات و پردازش: هیچ تفاوتی ندارد. نرخ نمونه‌برداری سخت‌افزاری سنسورها ثابت است (همان حدود $\sim 48.5\text{ (Hz)}$). بنابراین فرقی نمی‌کند گوشی را آرام حرکت دهیم یا ناگهانی؛ در هر دو حالت تعداد کل رویدادهای تولید شده در ثانیه، تعداد دفعات اجرای تابع فیلتر ([AirMouse_Filtering](#)) و تعداد پکت‌های ارسالی UDP کاملاً یکسان است و پیچیدگی ریاضی محاسبات ماتریسی تغییر نمی‌کند.
- از نظر مقادیر داده‌ها: در حرکت ناگهانی، مقادیر عددی شتاب خطی و سرعت زاویه‌ای ثبت شده در رویدادها بسیار بزرگ‌تر هستند و دیتای متفاوتی در شبکه جابجا می‌شود.

تفاوت در تأخیر (Latency Difference): تفاوت تأخیر محاسباتی خالص وجود ندارد، اما تفاوت در تأخیر درک‌شده توسط کاربر ([Perceived Latency](#)) و تأثیر نوسانات ([Jitter](#)) بسیار شدید است:

1. در حرکت آرام، خطای نوسان زمان سنسورها (مثلاً تغییر فاصله از ۱۳ تا ۲۹ میلی‌ثانیه) یا تأخیر پیکی شبکه (تأخیر ۲۶ میلی‌ثانیه‌ای UDP) اصلاً توسط چشم کاربر دیده نمی‌شود، چون جابجایی مکان‌نما روی صفحه به ازای هر فریم بسیار ناچیز است.
2. در حرکت ناگهانی و سریع، ماوس باید مسافت بسیار بزرگی را روی مانیتور در چند میلی‌ثانیه طی کند. در این حالت، اگر یکی از پکت‌ها به دلیل تأخیر سیستم عامل با ۲۶ میلی‌ثانیه تأخیر ارسال شود یا به خاطر ماهیت UDP در شبکه گم (Drop) شود، مکان‌نما روی صفحه

کامپیوتر به جای حرکت نرم، یکباره جهش می‌کند یا با لکنت حرکت می‌کند (Cursor Stuttering / Jank). بنابراین، تاخیر و خطا در حرکت ناگهانی بسیار مشهودتر و آزاردهنده‌تر لمس می‌شود.

جمع‌بندی و نتیجه‌گیری

در این پروژه، یک نمونه عملی از سیستم موس هوایی مبتنی بر گوشی هوشمند پیاده‌سازی شد. برنامه اندروید توانست داده‌های سنسورهای خام را دریافت و کالیبره کند، حرکت گوشی را به جابه‌جایی نسبی مکان‌نما تبدیل نماید و رویدادهای کلیک و اسکرول را تشخیص دهد. همچنین با استفاده از کلاس شبکه، داده‌های تولیدشده در قالب بسته‌های UDP به رایانه ارسال شدند و در سمت رایانه با کمک اسکریپت پایتون و کتابخانه PyAutoGUI به عملیات واقعی ماوس تبدیل شدند.

نتایج تحلیل عملکرد نشان می‌دهد که سیستم از نظر ساختار کلی توانسته نیازهای اصلی پروژه را پوشش دهد، اما کیفیت عملکرد آن به عواملی مانند نویز سنسورها، نرخ نمونه‌برداری، تأخیر شبکه، زمان‌بندی تردها و دقت کالیبراسیون وابسته است. استفاده از روش‌هایی مانند **deadzone**، **cooldown**، **gesture lockout**، انباشت زیرپیکسل‌ها و مکانیزم ACK برای رویدادهای حساس باعث شد حرکت موس پایدارتر و اجرای کلیک و اسکرول قابل‌اعتمادتر شود. در مجموع، پروژه یک پیاده‌سازی قابل قبول از یک سیستم سایبر-فیزیکی بلادرنگ ارائه می‌دهد و می‌تواند در نسخه‌های بعدی با بهینه‌سازی بیشتر فیلترها، کاهش بار **Main Thread** و بهبود رابط کاربری دقیق‌تر و روان‌تر شود.